Training–Rollout Mismatch in Low-Precision LLM Reinforcement Learning: A Survey

Zunhai Su[1], **Wei Wu**[2], **Jianchao Tan**[2], **and Ngai Wong**[1]
[1]The University of Hong Kong, [2]Meituan

Abstract: Low-precision rollout can reduce the cost of large language model reinforcement learning, but it can also create a mismatch between the policy that generates experience and the policy used to learn from it. This training–rollout mismatch can distort gradient estimation and policy updates, so faster generation alone does not establish more efficient learning. We synthesize twenty-nine studies through a common formulation that separates numerical execution differences from policy staleness. Methods are organized by where they intervene: aligning computation, preserving numerical fidelity and the ability to express parameter updates, correcting gradient estimation, or controlling policy updates. To interpret their evidence, we distinguish controlled numerical probes from online training diagnostics and relate reported measurements to learning quality and total system cost. The synthesis shows why numerical accuracy, agreement between execution paths, and useful policy improvement are distinct objectives, and identifies the information requirements and residual errors that govern method composition. Open challenges include predicting harmful mismatch, characterizing stateful behavior policies, and comparing learning quality at matched cost.

Keywords: quantized rollout; low-precision reinforcement learning; training–inference mismatch; importance sampling; quantization-aware training.

Literature snapshot: October 20, 2026.

Survey resources: https://github.com/ZunhaiSu/awesome-low-precision-rl

Updates, corrections, and collaboration enquiries: GitHub Issues.

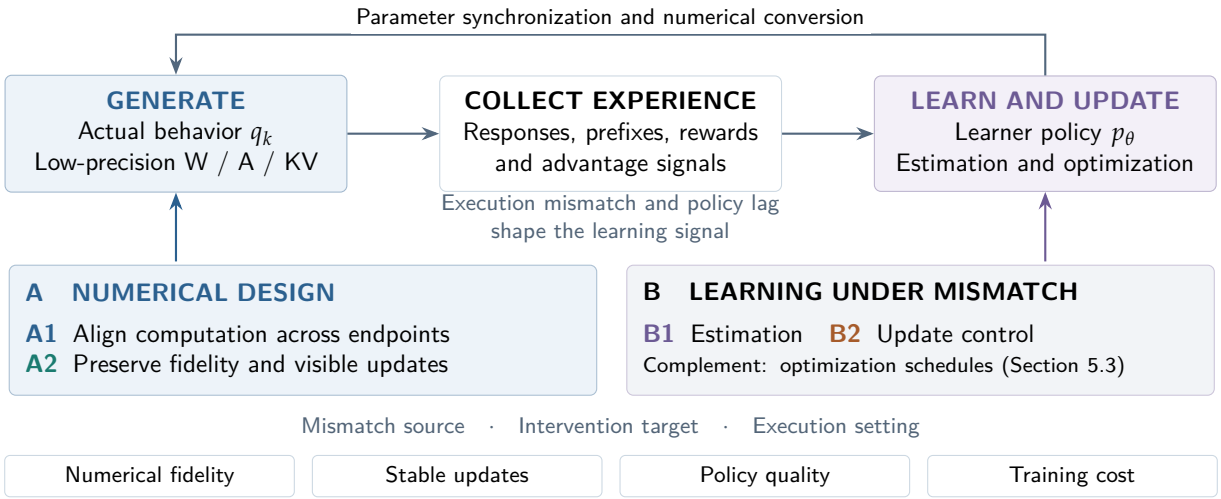


Figure 1: Survey at a glance. A addresses the numerical computations that produce disagreement; B manages its effects on learning from collected samples. Both act within the rollout–learning loop and can be combined. W, A, and KV denote weights, activations, and key–value cache.

Contents

1 Introduction and Scope **4**

1.1 Why Low-Precision Rollouts Complicate Policy Optimization 4

1.2 Scope and Related Work 4

1.3 Perspective and Contributions 4

2 Background and Problem Formulation **6**

2.1 Learner, Behavior, Proximal, and Reference Policies 6

2.2 Execution Mismatch versus Policy Staleness 7

2.3 A Component-Level Taxonomy of Interventions 8

2.4 Numerical Regimes and Contribution Roles 9

3 How Execution Mismatch Affects Learning **9**

3.1 Sources and Propagation of Execution Discrepancy 9

3.2 Distorted Estimation and Update Decisions 10

3.3 Feedback across Optimization Steps 11

3.4 Diagnosing and Measuring Mismatch 12

4 Numerical Design and Execution Alignment **14**

4.1 Aligning Learner and Rollout Computation 14

4.2 Controlling Quantization Error and Preserving Parameter Updates 16

4.3 Residual Mismatch and Limits of Alignment 17

5 Learning under Residual Mismatch **18**

5.1 Correcting Gradient Estimation 18

5.2 Controlling the Influence of Policy Updates 20

5.3 Adapting the Optimization Schedule 22

5.4 Compatibility, Bias, and Failure Modes 22

6 Evidence Synthesis and System Trade-offs **23**

6.1 Experimental Regimes . . . 24

6.2 Findings on Stability and Learning Quality . . . 25

6.3 Efficiency Gains and Implementation Costs . . . 27

6.4 Evidence Limits and Coverage Gaps . . . 28

7 Open Problems **29**

7.1 Predicting Harmful Mismatch . . . 29

7.2 Composing Numerical and Learning Interventions . . . 30

7.3 Precision, Staleness, and Long-Horizon Interaction . . . 30

7.4 Generalization and Comparable Evidence . . . 30

8 Conclusion **30**

A Surveyed Papers and Section Mapping **31**

B Source Selection and Version Handling **33**

C Evidence, Assumptions, and Transfer **34**

D Terminology **36**

E Reported Mismatch Measurements **37**

Reading pathways

Foundations	Methods	Evidence
Policy roles, execution mismatch, and its effects on learning.	Numerical interfaces, estimator correction, and update control.	Within-study findings, system costs, and open problems.
Sections 2–3	Sections 4–5	Sections 6–7

Appendix A: surveyed papers · Appendices B–C: sources and assumptions · Appendix D: terminology

Appendix E: mismatch measurements across core studies

1. Introduction and Scope

1.1. Why Low-Precision Rollouts Complicate Policy Optimization

In language-model reinforcement learning, the rollout engine generates the responses and histories used to train the next policy. Lowering its precision can reduce generation cost, but it also changes the distribution of training experience. Even when learner and sampler start from the same parameters, different rounding, kernels, or cached states can produce different token probabilities. This training–rollout mismatch is a numerical instance of training–inference mismatch (TIM).

A changed probability can alter which response is sampled, how it is weighted, and whether its gradient survives clipping. The update then changes later parameters and rollouts, coupling numerical execution to optimization. The objective is a capable policy learned at lower total cost; suppressing updates can stabilize training while also suppressing progress.

The survey follows the discrepancy from its computational source to its effect on learning. FlashRL's behavior-probability correction changes how samples are used; Jet-RL's coordinated precision flow changes the computations that produce their probabilities [1, 3]. Their contrast motivates a component taxonomy and a separate assessment of the evidence for each intervention.

Figure 2 separates numerical design (A), which changes policy computation, from learning interventions (B), which change how experience produces an update. A system can combine both. Appendix A maps all twenty-nine focal works to components and discussion locations.

1.2. Scope and Related Work

This survey focuses on numerical and execution mismatch in LLM reinforcement learning, especially quantized rollout and related low-precision training configurations. It covers a higher-precision learner paired with a lower-precision sampler, as well as low precision on both sides. Weight, activation, and key–value (KV) cache quantization affect different computational interfaces. Policy staleness enters where parameter changes interact with those interfaces.

The twenty-nine focal contributions comprise nineteen core method and system studies, nine supporting formulation, theory, or general mismatch studies, and one exploration boundary case. These roles describe their use in the synthesis: MiniRL, for example, develops the formulation and also evaluates FP8 rollout [14]. The literature snapshot is October 20, 2026. Appendix B describes source selection and version handling.

Broader surveys cover low-precision training, rollout strategies, and RL for reasoning models [33–35]. Primary studies distinguish precision regimes, mismatch sources, alignment and correction, or collection and cache design [3, 6, 10, 12, 17]. Here, intervention targets organize the methods and measurement conditions organize the evidence. Other quantization and rollout techniques enter where they explain this learning interface.

1.3. Perspective and Contributions

Our synthesis connects policy identity, intervention target, and empirical evidence. Explicit policy roles separate numerical mismatch, staleness, and optimization anchors. The taxonomy explains where methods act and how they combine; measurement analysis relates their effects to retained updates, policy quality, and total cost under the reported conditions.

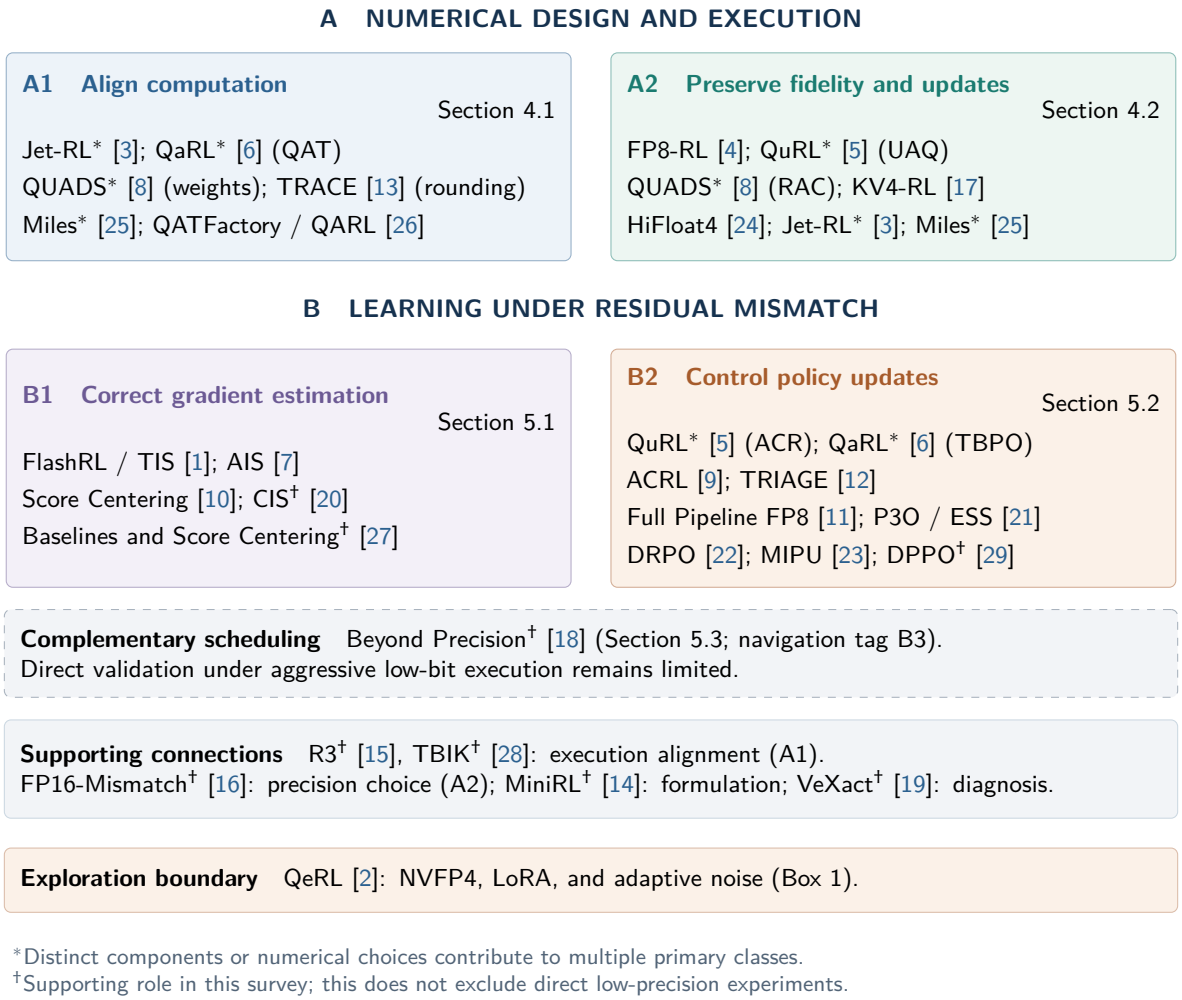


Figure 2: Component-level taxonomy of all twenty-nine focal studies. The four primary classes describe direct interventions; repeated names identify distinct components. Scheduling, supporting connections, and the QeRL exploration boundary are shown separately. Numerical regimes remain independent comparison axes. Appendix A gives the complete section mapping.

Sections 2–3 establish the background and mechanisms; Sections 4–5 compare interventions. Section 6 synthesizes quality and cost, leading to the research questions in Section 7.

2. Background and Problem Formulation

2.1. Learner, Behavior, Proximal, and Reference Policies

Let x be a prompt, $y_{1:T}$ a response, and $h_t = (x, y_{<t})$ its prefix. A policy is specified by both a parameter state and the computation used to evaluate it. Write $p_\theta(a \mid h)$ for the current learner policy and $p_k(a \mid h)$ for the same learner-side path at sampling snapshot θ_k.

The behavior policy $q_k(a \mid h)$ is the distribution that actually sampled the token. It includes the rollout engine, numerical representation, and sampling transformation. Temperature and vocabulary truncation therefore belong to its definition: a raw-model log-probability returned by an engine need not equal the probability after decoding transformations.

A proximal policy p_{prox} supplies the denominator of a policy-update constraint. A reference policy p_{ref} supplies a regularization target. These are different roles: a frozen pre-update learner may be proximal, while a much earlier policy is the KL reference. Neither is automatically the policy that generated every token. PPO and GRPO provide familiar contexts for these distinctions [30, 31]; mismatch-aware formulations require them to remain explicit [1, 14].

A running GRPO update. Consider a prompt for which the sampler generates G responses. A task-dependent scorer assigns reward R_i to response i; in RL with verifiable rewards, a correctness checker can supply this score without a learned reward model. A common group-relative advantage is $A_i = (R_i - \bar{R})/(\sigma_R + \epsilon_A)$, where $\bar{R}$ and σ_R are the group mean and standard deviation, and $\epsilon_A > 0$ stabilizes the normalization. For binary rewards $(1, 1, 0, 0)$, the advantages are approximately $(1, 1, -1, -1)$. The advantage is held fixed during optimization and is typically shared by the valid tokens of the response. Some variants omit standard-deviation normalization [31].

At the learning stage, a representative clipped objective on this fixed group is

$$\begin{aligned} \widehat{J}(\theta) &= \frac{1}{G}\sum_{i=1}^{G}\frac{1}{T_i}\sum_{t=1}^{T_i}\left[\ell(r_{i,t}, A_i) - \beta\mathcal{K}_{i,t}(\theta)\right], \\ \ell(r, A) &= \min\{rA, \text{clip}(r, 1-\epsilon, 1+\epsilon)A\}. \end{aligned} \tag{1}$$

where T_i counts valid response tokens, $r_{i,t} = p_\theta(y_{i,t} \mid h_{i,t})/p_{\text{prox}}(y_{i,t} \mid h_{i,t})$, ϵ sets the clipping interval, and β controls optional reference regularization. Here $\mathcal{K}_{i,t} = D_{\text{KL}}(p_\theta(\cdot \mid h_{i,t}) \| p_{\text{ref}}(\cdot \mid h_{i,t}))$ denotes the conditional KL; implementations can instead use sampled estimators. Maximizing the surrogate encourages positive-advantage actions and discourages negative-advantage actions. Clipping limits incentives for large changes relative to the proximal policy; reference KL provides a separate anchor.

After the update, weights are synchronized and converted for the next rollout. With matched execution and decoding, the pre-update learner can also be the behavior policy: $q = p_{\text{prox}}$. Quantized rollout breaks this identification even before an update. At $p_\theta = p_{\text{prox}}$, the proximal ratio is one, but p_{prox}/q can already differ from one. Section 5.1 shows how an explicit behavior correction can be separated from the update ratio. This loop locates the survey's interventions: numerical design changes the computations, estimator correction changes the use of sampled experience, and update control changes the accepted learning step.

MIPU targets improvement in the inference policy obtained after conversion and synchronization, since it supplies the next experience [23].

Table 1: Policies and quantities used throughout the survey. Execution paths are part of policy identity.

Symbol	Meaning
p_θ	Current learner policy under its specified execution path.
p_k	Learner-side policy evaluated at sampling snapshot θ_k.
q_k	Actual behavior policy that sampled the token.
p_{prox}	Policy used to define the proximal update ratio.
p_{ref}	Policy used for KL regularization; not the behavior denominator.
Δ_k	Same-snapshot log-probability difference, $\log p_k - \log q_k$.
d_θ	Current learner–behavior gap, $\log p_\theta - \log q_k$.
A_t	Advantage or other policy-gradient learning signal.
$s_\theta(a,h)$	Score vector, $\nabla_\theta \log p_\theta(a \mid h)$.
$c_t, \bar{c}_t$	Behavior-to-proximal correction and its truncated or masked form.
$r_t(\theta)$	Proximal update ratio, $p_\theta / p_{\text{prox}}$.
$W_\theta(y \mid x)$	Complete-response likelihood ratio, P_θ / Q.
N, sg	Valid-token count and stop-gradient operator in Equation (10).

2.2. Execution Mismatch versus Policy Staleness

At a fixed prefix and token with positive relevant probabilities,

$$\frac{p_\theta(a \mid h)}{q_k(a \mid h)} = \underbrace{\left(\frac{p_\theta(a \mid h)}{p_k(a \mid h)}\right)}_{\text{parameter change / lag}} \times \underbrace{\left(\frac{p_k(a \mid h)}{q_k(a \mid h)}\right)}_{\substack{\text{same-snapshot} \\ \text{execution / decoding}}}. \tag{2}$$

The first factor captures parameter change since collection, including asynchronous version lag; the factorization requires no independence assumption. The second compares execution at the sampling snapshot. It isolates numerical differences when decoding is matched and also includes decoding differences when temperature or top-p filtering changes. The intermediate policy p_k is a conceptual reference and need not be retained by the implementation. In partial rollout, a response can span collection boundaries, so $k = k(t)$ may vary across tokens. If caches survive synchronization, retained entries encode earlier computations. Their provenance records the versions and numerical paths that produced them; a current checkpoint identifier alone does not specify this behavior state. The notation q_k abbreviates that stateful rollout path.

Figure 3 separates these snapshot comparisons from the two policy roles used by the loss. A proximal anchor and a KL reference can be chosen independently of the actual sampler.

Two limiting cases make the distinction concrete. At the sampling snapshot, the learner-side parameter-change factor is one while execution mismatch can remain. With matched execution and decoding, the second factor can instead be one while policy lag remains. A numerical intervention and a staleness correction therefore solve different parts of the same ratio.

Probability ratios also require support conditions. A token observed from q_k has a defined denominator, but that alone does not establish that q_k covers every action with positive target probability. Truncation sampling can violate this condition. Similarly, a token-level change of measure is not automatically a correction for the prefix distribution. At fixed h,

$$\mathbb{E}_{a \sim q_k(\cdot \mid h)}\left[\frac{p_\theta(a \mid h)}{q_k(a \mid h)} f(a,h)\right] = \mathbb{E}_{a \sim p_\theta(\cdot \mid h)}[f(a,h)] \tag{3}$$

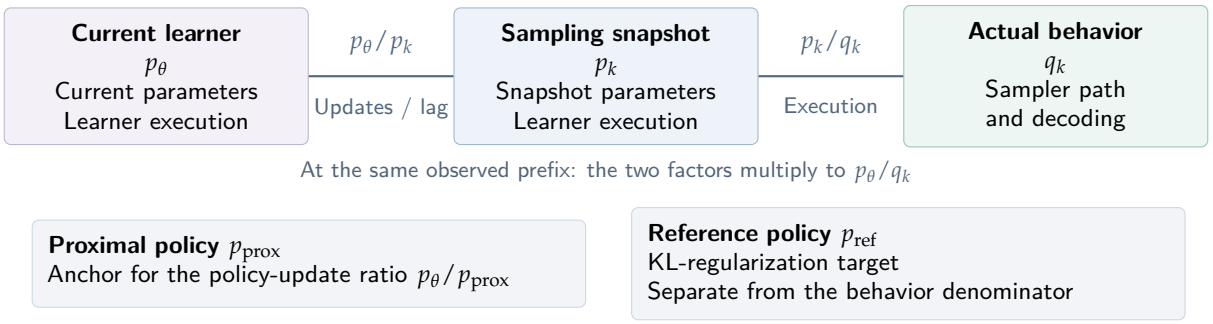


Figure 3: Policy identities and loss roles. Horizontal links denote the ratio comparisons in Equation (2), rather than data flow. Execution differences isolate numerical mismatch when decoding is matched. The lower cards identify separate roles in optimization.

holds under support and integrability assumptions. It says nothing by itself about whether the outer distribution over h is the desired one. It also does not undo changes to group-relative advantages induced by collecting different responses.

A token-level correction adjusts the action distribution at the observed prefix; it does not reconstruct histories that the sampler visited with a different frequency. Figure 13 gives a two-step example, and Appendix C states the corresponding full-response identities. The distinction matters for group-relative RL because changing the response distribution also changes the groups from which advantages are computed [10].

2.3. A Component-Level Taxonomy of Interventions

Three views describe a contribution: its *source* (why probabilities differ), *target* (what changes), and *setting* (tested precision, adaptation, and collection). Method classes follow the target.

A: numerical design addresses mismatch at its computational source. A1 aligns learner and sampler computations; A2 protects numerical information and the parameter updates that quantization can obscure. These components can operate on either endpoint. **B: learning under residual mismatch** addresses how disagreement affects learning from collected experience. B1 corrects gradient estimation; B2 constrains, selects, or accepts policy updates. B can preserve useful learning without making the two policies agree more closely. A intervenes in numerical execution; B manages the optimization consequences of residual mismatch (Table 2).

Classification follows a component's purpose. Both B1 and B2 can weight a loss: B1 derives a correction from a target expectation, whereas B2 constrains or selects updates. Truncation can bias a B1 estimator, and B2 controls can also change estimation. An importance ratio alone therefore does not determine the class. DPPO and DRPO contribute B2 trust-region geometry; Score Centering contributes B1 correction of a conditional score expectation [10, 22, 29].

The families are complementary. QaRL combines aligned QAT (A1) with TBPO (B2); QuRL combines update-aware quantization (A2) with adaptive clipping (B2); QUADS combines weight alignment (A1) with activation compensation (A2) [5, 6, 8]. An adopted correction such as TIS is recorded separately from a system's own contribution. Scheduling is complementary (B3).

Table 2: Intervention targets and decision questions. Figure 2 maps their associated studies.

Class	Decision question	What changes	Key dependency
A1	Can the two execution paths agree more closely?	Operators, representations, or realized numerical decisions.	Correspondence between computations.
A2	Is useful information preserved at low precision?	Scales, residuals, update resolution, or protected states.	Identification of sensitive information.
B1	How should behavior samples estimate a target gradient?	Importance weights or conditional score expectations.	Target distribution and available behavior information.
B2	Which updates should influence the next policy?	Constraints, contribution weights, or candidate acceptance.	Learning signal and control criterion.
B3	When and how strongly should learning proceed?	Complementary optimization schedule.	A signal for adaptation over time.

2.4. Numerical Regimes and Contribution Roles

Quantization maps values to a discrete set using a specified format, scale, and rounding rule. Weight quantization changes parameters that can be shared within a sampling version. Activation quantization acts on values recomputed during each forward pass. KV quantization acts on attention state retained across token positions. These targets differ in when their errors arise and how long those errors remain in the computation.

Post-training quantization converts a learned model without adaptation through the target quantizer. Quantization-aware training (QAT) models quantization during learning. Fake quantization quantizes and dequantizes operands while matrix multiplications can remain at higher precision; a straight-through estimator supplies surrogate derivatives through rounding, as in QUADS [8]. Native kernels perform the specified low-precision arithmetic directly. QAT specifies adaptation; speed and backward, master-weight, and optimizer precision must be reported separately.

Static PTQ and QAT do not exhaust the ways a quantized sampler can participate in RL. QuRL repeatedly quantizes the sampler while updating a higher-precision learner from the experience it generates; its parameters adapt indirectly to quantized rollout without an explicit learner-side QAT path [5]. Low-rank versus full-parameter updates and native versus simulated arithmetic are further independent choices. These distinctions avoid treating QAT, importance correction, and parameter-efficient adaptation as competing method labels.

Precision, adaptation, architecture, and collection protocol specify the experimental setting independently of a component's intervention target.

3. How Execution Mismatch Affects Learning

3.1. Sources and Propagation of Execution Discrepancy

Scales, rounding, accumulation, kernels, batch shape, and cached state can alter probabilities at the same nominal precision. FP16-based analysis examines precision selection [16]; R3 isolates routing differences that can occur without quantization [15].

Cross-TP deterministic inference isolates reduction order. TBIK coordinates reduction trees across tensor-parallel sizes to provide a matched-execution reference [28]. At lower precision, HiFloat4's ablations connect outlier-driven rollout activation underflow to degraded learning [24].

Shared-prefix probes compare conditional policies on fixed histories. Free-running comparisons also capture changed trajectories, where small errors can redirect generation.

Quantization can hide a learner update within one cell or amplify a small change near a boundary. QuRL addresses update visibility; TRACE addresses inconsistent rounding outcomes [5, 13]. Their targets depend on where evolving parameters lie relative to the quantizer's boundaries.

Backward arithmetic can change an update even when forward policies agree. Jet-RL specifies backward precision, and Miles offers a dequantized backward path to protect gradient fidelity [3, 25]. Interpreting low-precision failures therefore requires both policy-gap diagnostics and the learner's derivative computation.

Figure 4 organizes the resulting learning effects into relative influence, expected direction, and update selection. These routes can coexist, which explains why one aggregate disagreement statistic can miss the active mechanism.

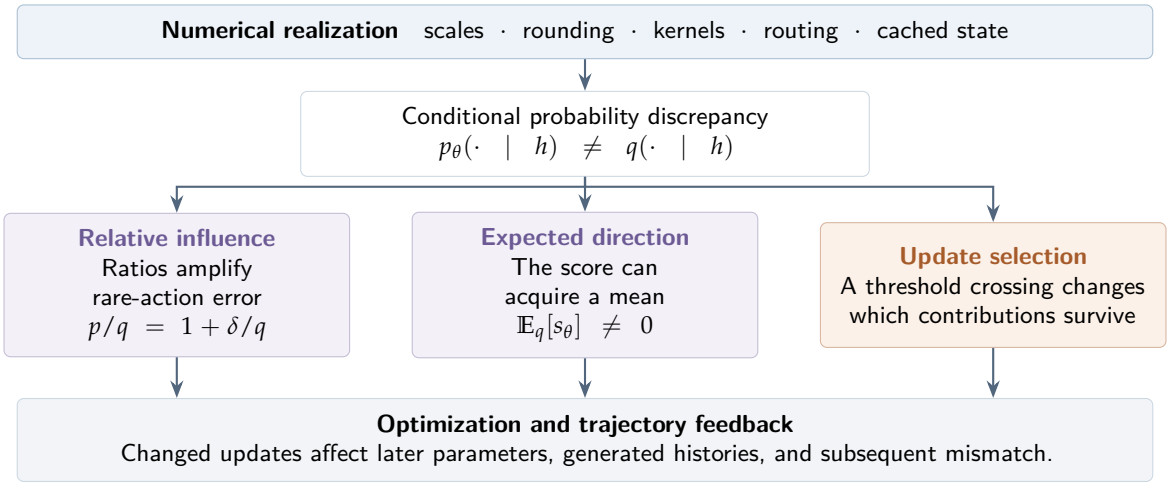


Figure 4: Three routes from execution discrepancy to learning: relative influence [1], expected score direction [10], and update selection [11, 12]. These effects can coexist and feed back through later updates and generated histories. Arrows denote propagation under the stated policy and estimator assumptions.

3.2. Distorted Estimation and Update Decisions

At an observed prefix, numerical disagreement can change an update through its weight, its expected direction, or the clipping branch it activates. Absolute probability error captures only part of this effect. Writing $p = q + e$ gives $p/q = 1 + e/q$: the same absolute perturbation has greater relative influence when q is small. Tails and confidence-conditioned statistics therefore reveal structure that a mean absolute error can hide. CIS uses this confidence dependence in its correction design [20].

DPPO makes a related distinction between relative ratios and moved probability mass [29]. A large ratio on a rare token need not represent a large distributional shift, while a moderate ratio on a common token can move substantial mass. This motivates separating the geometry of an update constraint from the numerical size of its sampled ratio.

Estimation can also acquire a systematic drift. For a normalized differentiable learner distribution, $\mathbb{E}_{a\sim p_\theta}[s_\theta(a,h)] = 0$. Under behavior sampling, the corresponding expectation need not vanish. At fixed h,

$$\mathbb{E}_q[As_\theta] = \mathrm{Cov}_q(A, s_\theta) + \mathbb{E}_q[A]\mathbb{E}_q[s_\theta]. \tag{4}$$

The expectations are conditional on the prefix and include the sampled action and any random continuation used to form A. Centering advantages across a prompt group need not center them at each prefix. Score Centering uses this distinction to motivate an additive correction [10]: it removes a first-moment drift term while leaving the behavior distribution itself unchanged.

Mismatch additionally affects decisions made by a surrogate objective. In a clipped policy-gradient objective, changing a ratio can change which branch is active and therefore whether a token contributes gradient. Thus, a perturbation can matter through a threshold crossing even when its absolute magnitude appears modest. The full-pipeline FP8 study isolates a failure involving excessive clipping of negative-advantage tokens [11]. This is a different intervention target from merely decreasing the variance of a sampled importance weight.

The sign dependence can be seen without assuming a particular quantizer. Let $\ell(r, A) = \min\{rA, \mathrm{clip}(r, 1-\epsilon, 1+\epsilon)A\}$, with $0 < \epsilon < 1$ and fixed A. Away from the two boundaries,

$$\frac{\partial \ell}{\partial r} = \begin{cases} A\,\mathbf{1}\{r < 1+\epsilon\}, & A > 0, \\ A\,\mathbf{1}\{r > 1-\epsilon\}, & A < 0, \\ 0, & A = 0. \end{cases} \tag{5}$$

Clipping depends on the advantage sign and the ratio being clipped. If the objective clips the combined learner–behavior ratio, $r = 0.7$ with $\epsilon = 0.2$ suppresses a negative-advantage token even at the sampling snapshot. This illustrates a different objective from the decoupled surrogate in Section 5.1.

Figure 5 shows the active branches: clipping the combined ratio can suppress an update that proximal-ratio clipping permits.

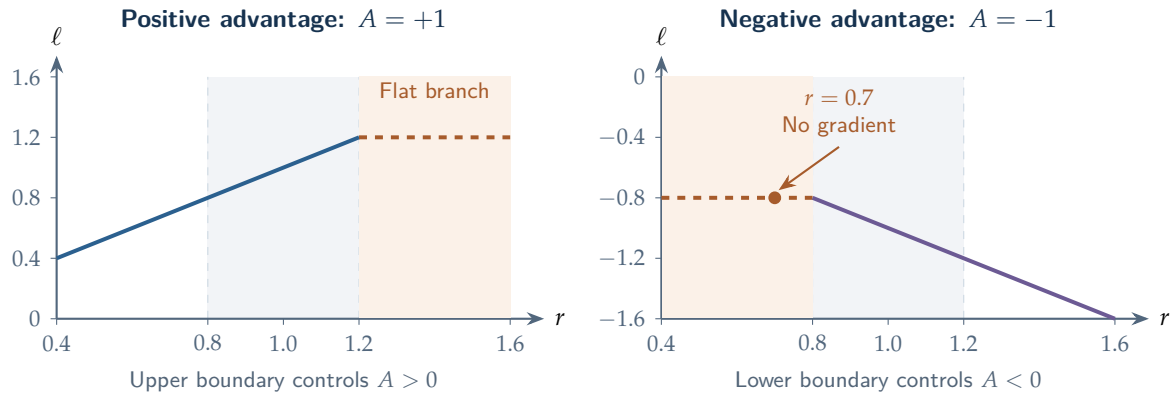


Figure 5: Signed clipping in Equation (5), with $\epsilon = 0.2$ and $A = \pm 1$. The gray interval is $[0.8, 1.2]$; amber segments are flat. At $r = 0.7$, a negative-advantage token contributes no gradient through this objective. Constructed example of the PPO surrogate with the specified ratio [30].

3.3. Feedback across Optimization Steps

The learning loop turns a local discrepancy into a dynamic process. Updates based on sampled responses change the learner; synchronization transfers those changes to the sampler; subsequent

rollouts come from the updated numerical realization. Stability depends on this feedback, not solely on whether one frozen batch has a small error.

The magnitude of a learner–behavior gap alone does not determine how an update changes it. In a simplified local picture, reducing a learner probability that is already below the behavior probability can enlarge their log gap. ACRL and TRIAGE use update-direction information in their controls [9, 12]. For a shared-parameter network, however, one token's update changes other probabilities too. The sign argument supplies local intuition; interactions through shared parameters determine the scope of any stability result.

Optimization scale adds another feedback channel. Beyond Precision studies the relationship between mismatch and optimization dynamics and proposes response-length-triggered learning-rate decay [18]. This connects numerical observations to the scale of optimization, while leaving transfer of the schedule to aggressive FP4 execution open.

3.4. Diagnosing and Measuring Mismatch

Mismatch measurements describe four levels: disagreement between computations, estimator reliability, changed update decisions, and learning outcomes (Table 3). Their interpretation depends on the policy pair, conditioning, and aggregation. This section defines the quantities; Appendix E maps the practices of all nineteen core studies and supplies reporting details.

Specify the comparison before the distance. A *controlled numerical probe* fixes parameters and token history to compare execution paths. An *online diagnostic* compares the current learner with recorded behavior, potentially including policy lag (Equation (2)). Figure 6 separates them. Raw-model, post-decoding, and recomputed scores remain distinct probability records; VeXact illustrates matched-execution attribution [19].

KV4-RL provides a particularly explicit conditional probe: it computes full-vocabulary $D_{\mathrm{KL}}(p_{\mathrm{BF16}} \| q_{\mathrm{quant}})$ at identical BF16-generated prefixes from the same checkpoint, before temperature or truncation [17]. It therefore measures conditional KV distortion, while leaving trajectory selection outside the metric. TRIAGE shows why conditioning also matters within one study: its 30B characterization replays sampled MoE routes, whereas its 4B run measures the ordinary learner–sampler gap. The paper explicitly avoids comparing their raw magnitudes [12].

Probability gaps and KL answer different questions. For a specified pair of normalized conditional policies p, q, let $d_t = \log p(a_t \mid h_t) - \log q(a_t \mid h_t)$ on sampled actions $a_t \sim q(\cdot \mid h_t)$. Common summaries include

$$G_{\mathrm{abs}} = \frac{1}{N}\sum_t |d_t|, \qquad \widehat{K}_1 = -\frac{1}{N}\sum_t d_t, \qquad \widehat{K}_3 = \frac{1}{N}\sum_t \left(e^{d_t} - d_t - 1\right). \tag{6}$$

Here N counts valid measured tokens, and G_{abs} removes the gap's sign. With common support and finite expectations, both KL estimators target conditional $D_{\mathrm{KL}}(q\|p)$ averaged over the observed prefixes. In finite samples, $\widehat{K}_1$ can be negative, while every $\widehat{K}_3$ summand is nonnegative. These sampled estimates differ from reverse-direction $D_{\mathrm{KL}}(p\|q)$, full-vocabulary evaluation, and KL to a regularization reference.

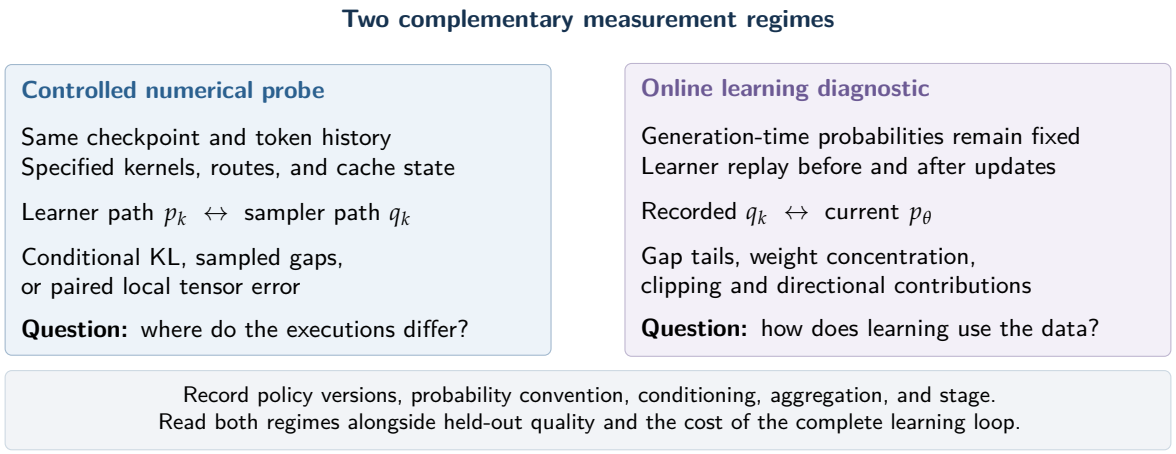


Figure 6: Controlled probes and online diagnostics target different comparisons [12, 17, 20]. Execution attribution depends on the factors held fixed; online gaps can include policy lag.

The distinctions appear in practice. TRIAGE and AIS report mean absolute log gaps; TRACE reports log-gap maxima and minima; QUADS tracks a maximum gap; QuRL reports behavior-to-proximal KL and a maximum ratio; MIPU explicitly uses sampled-token K3 [5, 7, 8, 12, 13, 23]. Extremes and tails reveal effects that a mean can hide, but depend on the size and composition of the measured population. Comparison requires matched definitions, populations, and reductions.

Table 3: Complementary levels of mismatch diagnosis. Each level answers a different question; outcomes require evidence connecting them to the preceding mechanisms.

Question	Representative measurements	Interpretation boundary
Where do computations disagree?	Shared-prefix KL; sampled-action gaps and tails; paired activation error [8, 12, 13, 17].	Specify checkpoint, prefix, routing, probability convention, and tensor location.
How reliable is the estimator?	Raw or capped weight concentration; ESS; conditional score mean; variance amplification [7, 10, 21].	Concentration and drift are different effects. A theoretical estimand is not automatically a logged statistic.
Which updates change?	Sign-separated clipping; directional mass and segment localization; candidate acceptance [11, 12, 23].	Requires the objective, fixed advantages, and the relevant update or synchronization stage.
Does learning remain useful?	Reward, held-out quality, entropy, failure time, and total cost.	Training outcomes assess consequences; they do not uniquely identify numerical mismatch.

Measure the route from disagreement to learning. Normalized empirical effective sample size (ESS) summarizes weight concentration:

$$\mathrm{ESS}_{\mathrm{norm}}(w) = \frac{(\sum_t w_t)^2}{N \sum_t w_t^2}. \tag{7}$$

P3O uses raw current-to-behavior ratios; AIS uses truncated weights [7, 21]. Their ESS values can differ on identical data. Uniform nonzero weights yield one even when their value is not one; high ESS therefore establishes neither calibration nor gradient fidelity. Score Centering instead corrects a conditional score mean (Section 5.1) [10].

Update diagnostics go further by asking which useful contributions survive. The full-pipeline FP8 study uses BF16 shadow passes to identify tokens whose active clipping branch changes under FP8, separated by advantage sign [11]. This counterfactual *over-clipping* differs from an ordinary clipping fraction. TRIAGE relates signed gaps to directional masses and local concentration, then audits the indicator against realized before/after updates [12]. These measurements connect discrepancy to learning decisions without assuming that every increase in disagreement is harmful.

Population and reduction are part of the metric. Token and response averages weight the data differently. Masks can lower a gap by removing difficult cases, and capping changes weight concentration. Policy versions, probability convention, conditioning, aggregation, and measurement stage determine the interpretation. Appendix E details the corresponding evaluation controls.

A diagnostic is informative when it links a specified discrepancy to an estimator or update effect and, in turn, to policy quality at a stated cost. Agreement matters through its effect on learning; minimizing every online gap is not the objective.

Box 1. Quantization and exploration

QeRL combines low-precision execution, low-rank adaptation, and adaptive quantization noise, making exploration an explicit part of the training design [2]. It occupies a boundary of this survey: the numerical perturbation changes the search process as well as agreement between computations. The reported benefit concerns structured quantization noise under low-rank adaptation and depends on the induced sampling distribution and the learner's adaptation capacity.

These mechanisms locate the survey's interventions: the computations that produce disagreement, the estimator that uses sampled experience, and the decisions that determine the next policy. Sections 4 and 5 develop interventions at these points in the learning loop.

4. Numerical Design and Execution Alignment

Numerical design can act on the learner, the sampler, or the interface between them. Alignment brings their computations closer at a shared interface; fidelity control preserves information that a low-precision representation would otherwise lose. The distinction matters because two approximate paths can agree without either becoming more accurate relative to a higher-precision model.

4.1. Aligning Learner and Rollout Computation

Jet-RL treats precision as a property of the computation graph shared by training and rollout [3]. It coordinates corresponding forward operators and quantization granularity instead of placing an FP8 sampler beside an otherwise unchanged learner. Its linear-layer forward and backward GEMMs use FP8 inputs, while gradients transported between operators remain BF16. The relevant alignment is thus more specific than a common format label and more substantial than learner-side fake quantization alone. Its results motivate coordinating the numerical path before asking a correction to absorb all cross-engine differences.

QaRL brings the learner forward closer to the representation used by quantized rollout through rollout-aligned QAT [6]. This numerical component is paired with TBPO, which controls sequence-level updates when low-bit generation produces problematic responses. The two components address different stages of the loop: QAT changes the probabilities computed by the learner, while

TBPO changes how sampled responses drive optimization. Their joint use makes QaRL an early example of alignment and update control within one method.

QUADS sharpens the distinction between weights and activations [8]. Weights synchronized across engines can share a quantize–dequantize path. Activations are recomputed from engine-dependent inputs, so applying the same coarse quantizer to both sides can send nearby values to different codewords. QUADS therefore fake-quantizes learner weights while retaining BF16 learner activations, then compensates selected rollout activation errors. Its asymmetric-versus-symmetric ablation shows why matching the list of quantized operands need not improve their realized agreement. The residual activation component is discussed in Section 4.2.

The shared object becomes progressively more specific: operator configuration, quantized representation, then the rounding outcome of a particular rollout.

TRACE aligns realized rounding decisions [13]. A shared FP4 format and scale can still send nearby learner and rollout activations to different codewords. TRACE records rollout-side quantization information and uses it to guide the learner's corresponding quantization site. With exact guidance and fixed rollout scales, it selects from the learner value's two neighboring codewords the one nearest the rollout codeword. Figure 7 illustrates why this can reduce cross-path discrepancy even when it slightly increases the learner's own quantization error.

The practical mechanism approximates this reference with compact code information. By default, TRACE caches one low code bit and the scales from the latter half of the layers, pairing records by trajectory, token position, and quantization site. The reconstruction uses the current learner activation as side information: after normalization by the rollout scale, it selects the nearest E2M1 magnitude whose retained code bits match the record, taking the sign from the learner. The adjacent learner candidates are then compared with this reference. Thus, the record narrows a local choice; it does not independently recover an arbitrary rollout value. Earlier layers can still be quantized without retaining this guidance. Reconstruction reliability consequently depends on the observed local adjacency of paired activations.

TRACE's main rollout quantizes expert W/A and softmax-layer KV to NVFP4, retaining BF16 elsewhere. The learner uses guided fake quantization at matching sites, with BF16 matrix multiplication, attention, and backward computation. Routing replay and token-level correction address residual differences; unpaired sites and policy lag remain sources of error [13].

System implementations extend alignment through conversion and synchronization. Miles coordinates checkpoint conversion, learner forward, rollout kernels, and live export through shared quantization rules; its MXFP8 and NVFP4 recipes retain precision exceptions and Beta status [25]. QATFactory's QARL combines native NVFP4 rollout with BF16 quantization simulation during training, then exports low-bit weights [26].

The shared information ranges from Jet-RL's numerical configuration to QAT representations and TRACE's rollout-specific decisions. Routing replay couples MoE choices [15]; TBIK coordinates reduction order [28]. Stronger coupling requires correspondence across tokens, modules, and parameter versions, while agreement still depends on the surrounding computation.

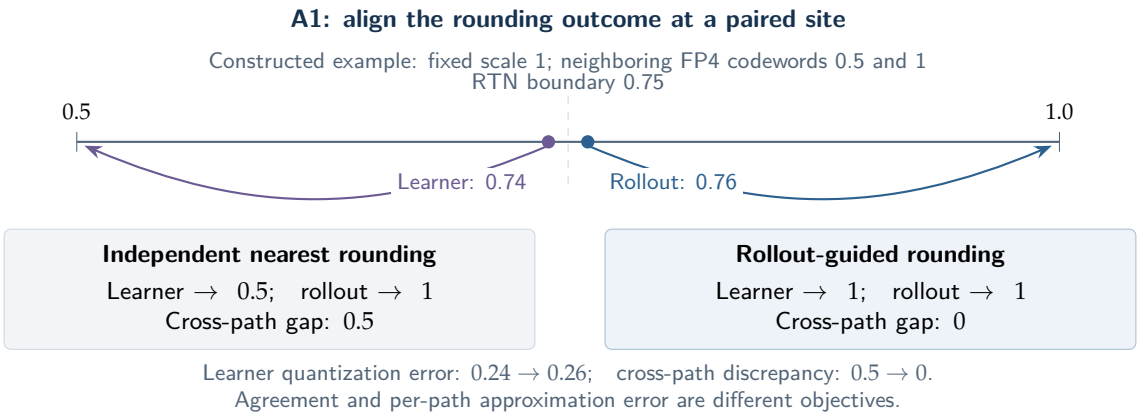


Figure 7: Rollout-guided rounding in a constructed fixed-scale E2M1 example. Agreement improves while learner approximation error increases slightly. Exact guidance applies at the paired site; TRACE's compact implementation approximates the reference [13].

4.2. Controlling Quantization Error and Preserving Parameter Updates

Alignment alone does not ensure that the sampler can express the changes made by training. For an illustrative deterministic quantizer Q, an update can satisfy

$$Q(w + \delta w) = Q(w), \tag{8}$$

even when $\delta w \neq 0$. Visibility depends on the distance to a quantization boundary and on scale behavior, as well as update magnitude. QuRL's Update-Aware Quantization addresses this gap through a one-time invariant rescaling before RL [5]. The real-valued identity $WX = (W/s)(sX)$ preserves a layer's initial output while changing the magnitudes seen by quantization and optimization. The method chooses the rescaling to improve update size relative to quantization noise. Preserving the initial real-valued function does not imply identical later optimization trajectories: the parameterization and gradients have also changed. UAQ therefore targets the sampler's ability to express learning, whereas QuRL's ACR changes which gradients reach the learner through clipping (Section 5.2).

Figure 8 shows a smaller update crossing a boundary while a larger one stays within its cell. Changing the scale moves these boundaries.

QUADS addresses the activation error left after learner-side weight alignment. Residual Activation Compensation restores selected high-error channels around the native W4A4 rollout computation [8]. Weight QAT and residual correction consequently change different quantities: the discretized weights shared with the learner and the rollout activation approximation. The component ablation separates the substantial recovery from weight alignment from the additional gain obtained by compensation. This is a concrete instance of A1 and A2 complementarity.

HiFloat4 / Rollout-ResQ targets rollout activation underflow through sparse residual correction [24]. Comparing learner and sampler precisions isolates a sampling-path error that raising learner precision alone does not resolve. The FP4 experiments use simulation, so they characterize learning under the simulated arithmetic; native-FP4 acceleration remains unmeasured.

FP8-RL maintains numerical fidelity as the policy changes through blockwise W8A8 rollout, scale recalibration for FP8 attention and KV execution, and dynamic weight synchronization [4]. It integrates these components with adopted TIS/MIS correction. Its v2 experiments also explore end-to-end FP8, so learner precision varies across the paper's experimental settings.

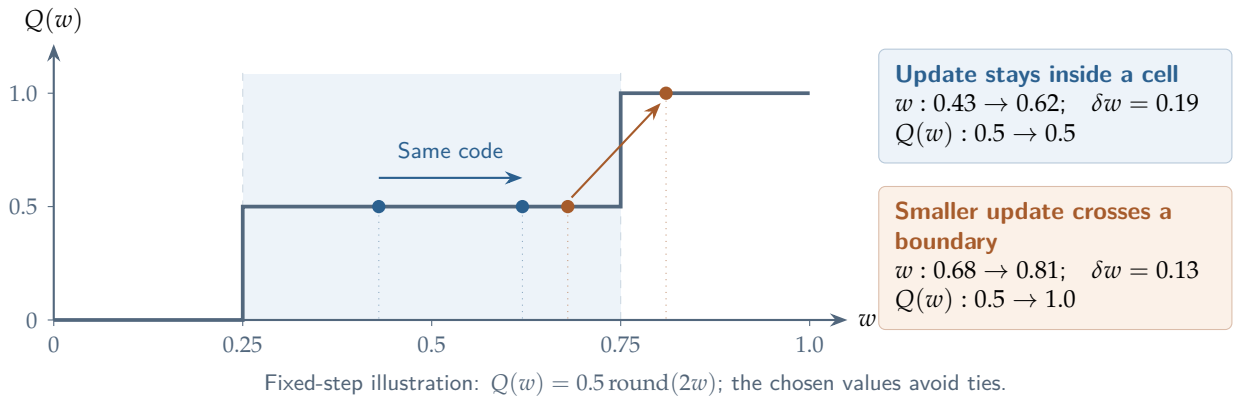


Figure 8: Constructed illustration of update visibility (Equation (8)). The blue update stays inside a quantization cell while the smaller amber update crosses a boundary. Changing scales moves the boundaries.

Miles exposes fidelity choices within an integrated precision recipe: selected tensors remain in BF16, and an optional dequantized backward path trades training throughput for gradient stability without changing the quantized forward values [25]. Such exceptions belong to the numerical configuration; they do not imply that every operation uses the headline format.

KV4-RL allocates fidelity across positions instead of raising precision uniformly [17]. INT4 KV storage is supplemented by bounded INT8 protection for attention-sink and recent positions. Its retained TIS recipe makes the distinction between sampler fidelity and conditional correction especially clear: protecting the cache changes the data-generating computation even when the correction is held fixed. The experiments use colocated, quota-based partial collection, with generation and optimization alternating; responses may span collection boundaries.

Rescaling protects update resolution, compensation reduces activation error, recalibration follows distributional change, and positional protection preserves sensitive cache states. Scaling, rotation, and residual correction describe useful quantization tools [24]. Their suitability for RL also depends on when they are chosen and how they track the evolving policy.

4.3. Residual Mismatch and Limits of Alignment

A shared quantizer can coexist with different activations or surrounding kernels. Alignment is therefore defined at a specified interface, snapshot, and history; fidelity separately concerns approximation to a higher-precision computation.

Residual differences connect numerical design to learning correction. A more faithful sampler may still lag behind the learner, and conditional agreement at observed prefixes does not restore a response distribution changed by earlier decisions. A learning intervention addresses how these remaining differences affect estimation or the resulting update.

Maintaining a shared numerical interface requires information as well as arithmetic. Operator choices require coordination, QAT depends on matching quantizer semantics, and rounding guidance requires storage and correspondence with learner states [3, 6, 8, 13]. Fidelity control instead uses calibration, compensation, or selective execution to preserve sensitive information [4, 5, 17]. Table 4 compares these dependencies and the discrepancies each leaves unresolved.

Table 4: Numerical strategies: shared information, execution dependencies, and residual differences.

Strategy	Shared or protected object	Information dependency	Residual discrepancy
Precision flow	Corresponding operator precision and granularity	Coordinated numerical configuration	Engine and surrounding-kernel differences.
Aligned QAT	Learner representation corresponding to rollout	Matching quantizer semantics	Native versus simulated arithmetic; unmatched activations.
Guided rounding	Realized rollout quantization decisions	Stored and matched rollout guidance	Unshared modules and downstream propagation.
Fidelity control	Scales, update resolution, residuals, or protected states	Calibration, compensation, or selective execution	Other execution paths and policy staleness.

Residual disagreement makes the use of collected experience the next design choice: which expectation to estimate and which policy changes to permit.

5. Learning under Residual Mismatch

Residual mismatch enters learning through both estimation and optimization. Estimator correction changes how behavior samples represent a target expectation; update control regulates their influence; scheduling changes the scale of adaptation over time. These objectives lead to different trade-offs among bias, variance, progress, and stability.

5.1. Correcting Gradient Estimation

Multiplicative correction and truncation. Importance sampling supplies a conditional change of measure under the assumptions of Equation (3). Practical surrogate objectives usually distinguish the mismatch correction from the proximal update ratio. With frozen p_{prox} and recorded behavior q, define

$$c_t = \frac{p_{\text{prox}}(a_t \mid h_t)}{q(a_t \mid h_t)}, \qquad r_t(\theta) = \frac{p_\theta(a_t \mid h_t)}{p_{\text{prox}}(a_t \mid h_t)}. \tag{9}$$

A representative decoupled surrogate is

$$\widehat{L}(\theta) = \frac{1}{N} \sum_t \text{sg}(\bar{c}_t) \min[r_t(\theta) A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t], \tag{10}$$

where $\bar{c}_t$ is a possibly truncated correction, the sampled advantages are held fixed, and sg denotes no differentiation through the correction weight. Before truncation and clipping, $c_t r_t = p_\theta / q$: the product compares the current learner with the actual sampler. Keeping the two factors separate makes their purposes explicit. Bounding c_t limits the correction for how a token was collected; clipping r_t limits the incentive to change its probability during the update. Individual objectives differ in how they combine these operations. An upper-truncated correction, for example, takes $\bar{c}_t = \min(c_t, C)$, whereas masking can discard selected ratios. Here N counts valid sampled tokens. Response-level averaging and length normalization assign different relative weights, so the reduction rule is part of the estimator being compared.

FlashRL's truncated importance sampling (TIS) provides a basic bounded correction using actual rollout probabilities [1]. MiniRL exemplifies the mismatch-aware distinction between behavior and proximal policies underlying this notation [14]. FP8-RL adopts token-level TIS and masked importance sampling (MIS) variants, while KV4-RL retains TIS alongside cache protection [4, 17]. The system studies show how the correction operates alongside other numerical components. Truncation bounds extreme influence while changing the expectation; recording an actual behavior denominator supplies the required probability semantics, but does not remove that truncation bias.

Adapting the amount and location of correction. Adaptive importance sampling (AIS) makes correction strength depend on both necessity and reliability [7]. Its effective multiplier has the form $1+\alpha(\bar{w}-1)$: $\alpha=0$ retains the uncorrected estimator, while $\alpha=1$ applies the truncated importance weight $\bar{w}$. A mean absolute log gap indicates whether correction is needed; capped-weight ESS and advantage-weight variance amplification indicate whether it is reliable. A large mismatch can therefore coexist with weak correction when the weighted estimator is too concentrated or noisy. AIS adapts an estimator mixture rather than the precision or clipping threshold.

Calibrated importance sampling (CIS) changes where truncation acts. Its log-odds displacement separates confidence from cross-engine perturbation: the same displacement can induce very different ratios at low- and high-probability actions [20]. A confidence-dependent cap follows this representation. AIS and CIS thus adapt different objects—mixture strength and truncation location—using different diagnostics. CIS supplies general TIM evidence from BF16 cross-engine experiments; transfer to an FP4 numerical path remains to be evaluated.

Additive score correction. Score Centering addresses a different object from the sampled-token importance weight [10]. Here the *score* is the parameter-gradient vector $s_\theta(a,h)=\nabla_\theta \log p_\theta(a \mid h)$, not the reward assigned to an answer. Its mean is zero under the learner distribution, but can be nonzero under the actual behavior distribution. In Equation (4), that conditional mean produces an additional term proportional to the conditional mean advantage. Group-centered rewards need not remove it because prompt-level centering does not center the signal at every generated prefix.

Score Centering replaces the score at a prefix with

$$\widetilde{s}_\theta(a,h)=s_\theta(a,h)-\mathbb{E}_{b\sim q(\cdot|h)}[s_\theta(b,h)]. \tag{11}$$

With the exact expectation, its mean under q vanishes and the drift term in Equation (4) is removed [10]. The remaining covariance is still measured under q, so this is not full recovery of an on-policy gradient. The reported implementation uses sampler top-k log-probabilities and models the omitted tail using the learner distribution; its default is $k=128$ [10]. The omitted mass therefore enters through a modeled tail rather than being discarded. The coefficients used to form the correction are detached, and its fidelity depends on that behavior reconstruction.

Figure 9 gives a one-parameter example. Changing the gradient's conditional mean does not require changing the sampler or forcing the learner probabilities to equal it. This separates a correction to the estimator from direct numerical alignment. When combined with importance weighting, the centered object must be the corresponding weighted score; an arbitrary weighted loss with the original unweighted center need not have the intended cancellation [10].

The covariance analysis of baselines and score centering makes estimator granularity central to their composition [27]. For the sequence-level regularized importance estimators studied there,

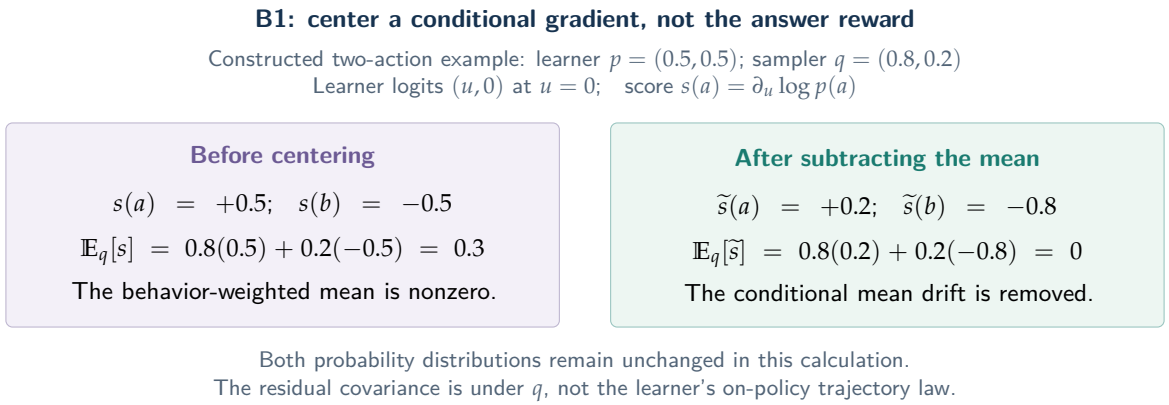


Figure 9: Constructed scalar-logit example of score centering [10]. Sampler probabilities weight the derivatives in Equation (11). Centering changes the conditional gradient mean while leaving both distributions fixed; practical top-k reconstruction adds an approximation.

the group-mean baseline already supplies the centering benefit. At token level, the same baseline generally leaves bias under regularized ratios. The useful centering construction therefore depends on the estimand, weighting, and reduction rule.

The information requirement follows the target. TIS-like correction, including the adaptations in AIS and CIS, uses sampled-action probabilities. Score Centering also reconstructs a conditional expectation over actions: sampled-token log probabilities alone cannot supply this first moment.

5.2. Controlling the Influence of Policy Updates

Update control spans five decisions: where to place a boundary, how to adapt it to batch reliability, which divergence geometry to use, which signed contributions to retain, and whether to accept a synchronized candidate. Table 5 separates these decisions from their aggregation units.

Table 5: Update-control mechanisms within B2, with their decision signals and aggregation units.

Decision	Signal or constraint	Unit	Representative components
Boundary placement	Mismatch-aware, trust-band, or reference-calibrated clipping.	Token or response.	QuRL ACR; QaRL TBPO; Calibrated Clipping.
Batch adaptation	ESS determines cap and regularization.	Batch statistics, applied through the objective.	P3O.
Constraint geometry	Divergence-based boundary or smooth regularizer.	Token-level probability shifts.	DPPO; DRPO.
Directional influence	Advantage and discrepancy determine retained or corrective contributions.	Tokens and segments within responses.	ACRL; TRIAGE.
Candidate acceptance	Assessment after inference-side synchronization.	Whole candidate policy.	MIPU.

Restoring or reshaping update constraints. QuRL's Adaptive Clipping Range (ACR) responds to the relation between proximal and behavior probabilities [5]. Its concern is that a fixed clipping rule can interact poorly with the mismatch already present at collection. ACR changes the permitted update region; UAQ, discussed in Section 4.2, instead changes whether the quantized sampler can express the resulting weight update. These two mechanisms distinguish failure to learn an update from failure to transmit it through quantization.

QaRL's TBPO constrains whole responses through negative-sample treatment and a sequence-level trust band [6]. Full-pipeline FP8 instead calibrates clipping against BF16: it matches the lower quantile on negative-advantage tokens, then adjusts the upper boundary to rebalance signed contributions [11]. Periodic reference passes amortize calibration of the update boundary.

Clipping statistics depend on advantage sign, reduction, and comparison policy. A wider region is useful when its additional contributions improve learning.

Adapting control to batch reliability. P3O uses normalized effective sample size (ESS) of the batch's policy ratios to adapt a score-weight cap and off-policy regularization [21]. Batch reliability thus determines how strongly the update is constrained. Its FP8-rollout experiment studies numerical mismatch directly, while the broader comparisons show dependence on the model and collection setting. AIS adapts the mixture of gradient estimators; P3O uses batch reliability to set update constraints. Similar adaptive machinery consequently serves a different target.

Choosing the constraint geometry. DPPO introduces a divergence-based trust region [29]. DRPO retains Binary-TV geometry but replaces hard masking with a smooth regularizer that supplies corrective gradients outside the boundary [22]. DRPO evaluates FP8-rollout-only and end-to-end FP8 MoE settings, extending the available precision evidence for this update geometry.

Controlling disagreement through update direction. ACRL steers disagreement through advantage-guided token reweighting controlled by response-level discrepancy from a reference [9].

TRIAGE connects the advantage sign to the learner–sampler log gap [12]. Using $d_\theta = \log p_\theta - \log q_k$ in this survey's notation, the product $A_i d_{\theta,i,t}$ identifies the sign of a simplified token's own mismatch-energy contribution. Cross-token coupling and optimizer history can change the realized motion, so this is a local directional indicator. Both $A_i > 0, d_\theta > 0$ and $A_i < 0, d_\theta < 0$ are locally amplifying. TRIAGE intervenes selectively on the latter: increasing the probability of a reward-positive response can be legitimate learning even if its gap grows.

The gate first diagnoses 64-position segments of negative-advantage responses from their mean gap and deep negative tail. Within an eligible risky segment, only negative-gap tokens receive an attenuated raw weight $b_{i,t}$. The weighted response loss is normalized by the sum of its weights. Relative to ordinary response averaging, its effective multiplier is therefore $b_{i,t}/\bar{b}_i$, not simply $b_{i,t}$. Normalization reallocates contributions: for illustrative raw weights $(0.3, 1, 0.3, 1)$, the effective multipliers are approximately $(0.46, 1.54, 0.46, 1.54)$. If every weight is equal, the normalization cancels it. This controls relative influence, not the total gradient norm.

Repair targets positive-advantage responses with sufficiently negative mean gaps in 128-position segments. A one-sided pseudo-Huber penalty raises learner log probabilities. Its derivative bounds

the additive coefficient, not the full gradient or every token's gap. All valid tokens in an active segment share that coefficient, including positive-gap tokens. Gating and repair thus address different signed contributions [12]. The tested recipe uses native W4A4 forward on both endpoints and BF16 KV. Applying it to KV4 remains an untested transfer.

Accepting whole updates in the inference policy. MIPU constructs candidates relative to the sampler, then assesses whether a synchronized candidate should become the next inference policy [23]. This separates candidate construction from acceptance after numerical conversion. The practical inference-gap proxy supports acceptance or rollback in the evaluated FP8 runs; a general guarantee of monotonic improvement would require stronger conditions.

The decision and its unit must be read together. Local rules can reveal concentrated risk but rely on noisier statistics; aggregation improves reliability while obscuring location. Whole-update acceptance asks a further question: whether a favorable learner-side candidate remains useful after conversion to the inference policy.

5.3. Adapting the Optimization Schedule

Beyond Precision adapts the learning-rate schedule to the evolution of optimization and mismatch [18]. Scheduling changes the scale of successive parameter updates, complementing numerical alignment and sample-level control. Direct evidence under aggressive quantization remains limited. Its value depends on preserving quality within the same budget, since smaller steps can delay progress as well as divergence.

5.4. Compatibility, Bias, and Failure Modes

Two interventions are complementary when they address residual effects left by one another. Numerical and learning components often meet this description: weight alignment can leave activation error, and a more faithful sampler can still generate off-policy data. The tested combinations in QaRL, QuRL, and QUADS illustrate distinct forms of complementarity [5, 6, 8].

Estimator compatibility depends on weighting, centering, masking, and reduction. Token- and sequence-level regularized estimators can require different centers [27]. Figure 10 distinguishes this estimation target from the contributions selected by update control.

Score Centering removes a conditional score mean; TRIAGE changes selected contributions using advantage and log-gap signs. Their distinct targets motivate composition, whose joint effects remain an open experimental question [10, 12].

Composition requires consistent policy records, a well-defined gradient, and useful learning at acceptable total cost. In particular, an importance-weighted score needs its corresponding weighted center [10]; successive masks can suppress the same examples, and guidance or reference passes add work. Table 6 identifies combinations whose joint behavior has been studied.

Table 6: Studied component combinations, their intervention targets, and the residual effects addressed.

Study	Combined components	Mechanistic connection
QuRL [5]	Update-aware quantization (A2) and adaptive clipping (B2).	Make parameter changes visible to the sampler and reshape the learner's update constraint.
QaRL [6]	Aligned QAT (A1) and TBPO (B2).	Reduce representational disagreement, then constrain problematic response-level updates.
QUADS [8]	Weight QAT (A1) and residual activation compensation (A2).	Weight alignment leaves activation error for a separate numerical component.
KV4-RL [17]	Selective cache protection (A2) with retained TIS.	Improve the data-generating path while holding a bounded conditional correction in the recipe.
TRACE [13]	Guided QAT (A1), routing replay, and token-ratio correction.	Couple rounding outcomes while separately addressing route choices and residual off-policy effects.
Score Centering [10]	Centering composed with TIS or MIS (B1).	Center the corresponding weighted score; reconstruction and regularization determine the remaining bias.

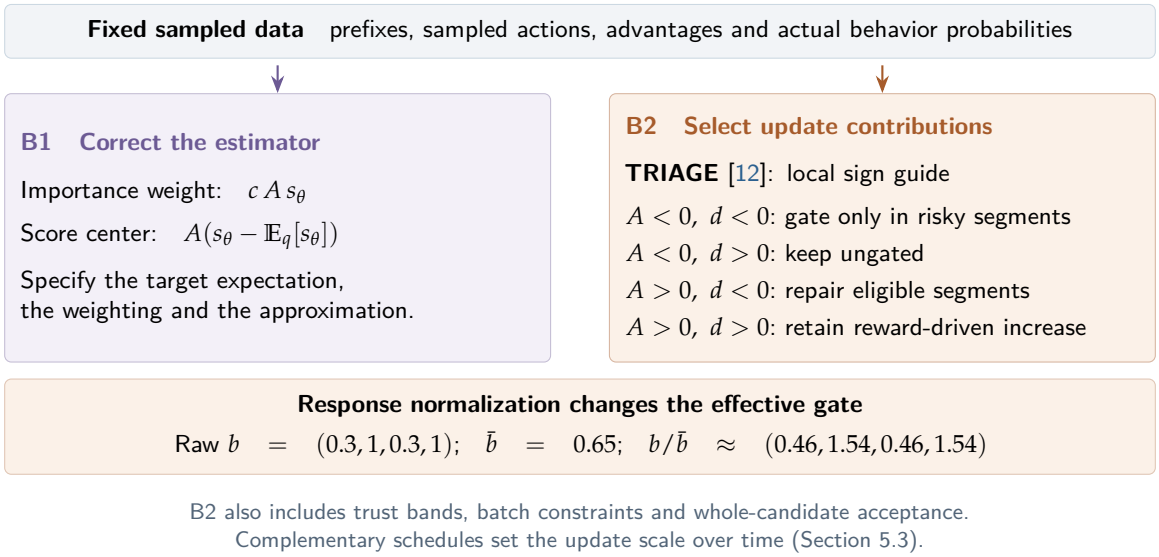


Figure 10: Estimator correction and update control. TRIAGE [12] uses $d = d_\theta$ with segment-level conditions. Repair acts on positive-advantage, negative-mean segments and includes all their valid tokens. Raw weights illustrate normalization, rather than a diagnosed segment; shared parameters limit token-wise predictions.

6. Evidence Synthesis and System Trade-offs

Three findings recur: matching formats is weaker than matching computations; useful control preserves informative updates; and generation gains change when measured over the complete RL loop. The following comparisons connect these findings to each study's model, numerical path, evaluation rule, and implementation cost.

6.1. Experimental Regimes

Asymmetric learner–sampler precision exposes differences between numerical representations. Low precision on both sides makes the consistency of their execution paths and the resulting update especially salient. KV quantization acts on retained state, whereas weight and activation quantization alter other parts of computation. Low-rank adaptation and full-parameter RL also provide different degrees of freedom for responding to these changes.

AIS evaluates diffusion-based LLaDA alongside autoregressive Qwen models [7]. Its autoregressive experiments connect directly to the token-level formulation used here. Interpreting the denoising results requires likelihoods defined for that generation process, so those experiments broaden the empirical scope without validating the autoregressive factorization for diffusion.

Evaluation precision answers a separate question from training precision. A common BF16 evaluator, as used in KV4-RL, compares the parameters learned from different rollout distributions while holding the evaluation path fixed [17]. Evaluation through the intended low-precision sampler also includes the numerical realization of those parameters, as in TRACE's FP4 policy results [13]. Evaluation precision is therefore part of the definition of policy quality.

Architecture and collection protocol further condition conclusions. MoE execution introduces discrete route choices; cache-heavy long responses expose different runtime bottlenecks from shorter generations; partial collection permits a response to contain tokens sampled under different parameter versions. These differences affect both the mismatch mechanism and the cost of intervention.

Miles adds production-scale agentic system evidence, whereas QATFactory isolates a smaller NVFP4 QARL workload [25, 26]. Native low-precision arithmetic, simulated quantization, and BF16 execution on quantized operands are distinct regimes. In particular, HiFloat4 reports simulated FP4 RL rather than measured native-FP4 training [24].

Table 7 summarizes four evaluated learner and rollout configurations. Appendix B maps coverage across numerical, architectural, and collection regimes (Table 12).

Table 7: Representative learner, rollout, and collection configurations. QAT specifies adaptation; forward, backward, and optimizer precision are separate attributes.

Study	Learner path	Rollout and collection setting
TRACE [13]	Guided fake-quantized forward; BF16 backward.	NVFP4 W4A4 on routed experts and KV4 in softmax attention; other modules BF16. Asynchronous collection.
TRIAGE [12]	Native NVFP4 W4A4 forward.	Native W4A4 forward; BF16 KV in the documented characterization. Synchronous math RL; no reported KV4 experiment.
Score Centering [10]	BF16 learner; distributional reconstruction supplies the correction.	Quantized-sampler stress tests include FP4 KV and INT8 sampler with INT4 KV. Policy-lag tests form a separate setting.
KV4-RL [17]	BF16 learner.	INT4 KV with bounded INT8 protection. Quota-based partial collection; generation and learning alternate. This is a cache-focused experiment.

The following analysis connects these execution choices to learning outcomes (Table 9).

6.2. Findings on Stability and Learning Quality

Component comparisons are most informative when they connect an intermediate effect to the resulting learning trajectory. Figure 11 identifies the baselines evaluated in TRACE and TRIAGE; Table 9 summarizes their results alongside Score Centering and KV4-RL. Appendix E compares diagnostic practices across all nineteen core studies.

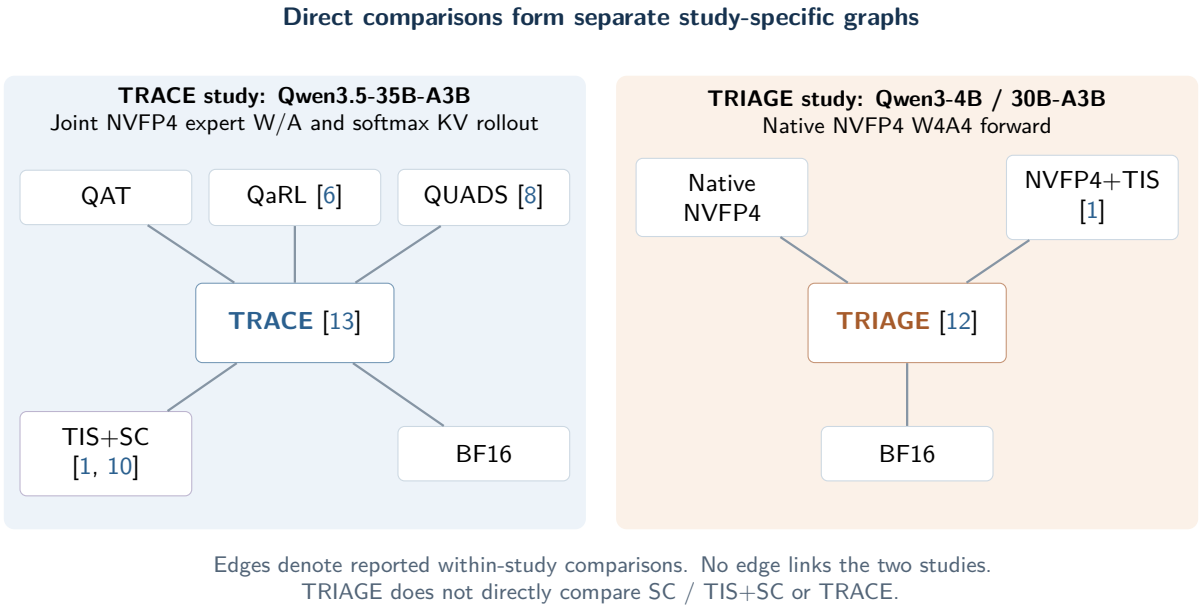


Figure 11: Study-specific comparison graphs for TRACE and TRIAGE [12, 13]. BF16 references belong to their respective settings; edges denote comparisons. Table 9 gives outcomes and checkpoint conditions.

Read agreement alongside learning progress. Entropy, reward, and probability gaps describe different aspects of a training trajectory (Table 8). Section 3.4 defines mismatch measurements; their joint evolution reveals whether closer agreement accompanies useful learning.

Table 8: Reading learning trajectories. Definitions illustrate common conventions; the source policy and reduction determine the logged quantity.

Quantity	What it measures	What it does not establish
Policy entropy H	$-\sum_a p(a) \log p(a)$ for a specified next-token policy, subsequently aggregated over prefixes.	A lower value need not indicate better learning. A label such as *entropy loss* does not establish that entropy regularization is active.
Mean reward $\bar{R}$	$B^{-1} \sum_i R_i$ over B sampled responses. With strictly binary correctness reward, it is the sampled correctness rate.	It is not the group-normalized advantage, a held-out benchmark score, or an invariant scale across reward functions.
Mean absolute gap G	Magnitude of learner–behavior disagreement on valid sampled tokens; Equation (6).	It removes sign and local concentration; it is neither KL divergence nor proof of estimator fidelity or policy quality.

TRIAGE's 30B experiment makes this distinction concrete: TIS prevents the aggregate gap from diverging over the measured horizon, yet its later reward and downstream scores remain below the

full method [12]. Useful reward-driven updates can also increase signed gaps. The relevant question is which learning contributions survive, rather than whether every discrepancy decreases.

A diagnostic also has a role within the method: AIS uses a gap in its controller, KV4-RL evaluates cache distortion through conditional KL, and TRIAGE uses signed, localized gaps to motivate an intervention [7, 12, 17]. Evidence is strongest when a component changes the proposed intermediate effect and preserves learning, including outcomes beyond the metric it directly controls.

Correction and mismatch co-evolve during learning. QuRL observes disagreement beyond an initially stable period, with ACR and UAQ addressing different changes in clipping and update visibility [5]. AIS adapts the correction mixture, ACRL regulates discrepancy through update influence, and Score Centering removes a conditional estimator drift term [7, 9, 10]. Each intervention changes the policy and hence the mismatch encountered later. P3O's Qwen3-8B FP8 comparison similarly links batch-adaptive control to preserved later performance within the evaluated horizon [21]. Correction and discrepancy therefore evolve jointly as the sampled policy changes.

Shared computation matters more than nominal symmetry. Jet-RL couples precision flow; QaRL, QUADS, and TRACE couple representations or realized rounding decisions [3, 6, 8, 13]. QUADS separates two effects: asymmetric weight QAT recovers much of the performance loss, and rollout activation compensation further narrows the gap. Its asymmetric-versus-symmetric comparison shows why nominal symmetry alone does not define effective alignment. HiFloat4's simulated precision ablations also locate a residual error in rollout activations [24]. FP8-RL follows a changing policy through calibration and synchronization, while QATFactory compares learning in the target representation with quantizing only the learned endpoint [4, 26]. Together, these comparisons support examining what is actually shared—operators, weights, scales, or rounding outcomes—before interpreting a claim of aligned precision.

Preserving useful updates matters beyond reducing average gaps. The full-pipeline FP8 study connects excessive lower-side clipping of negative-advantage tokens to pathological training dynamics, whereas TRIAGE uses local and direction-dependent information to select interventions [11, 12]. DRPO preserves corrective gradients outside its trust-region boundary; MIPU instead tests a whole candidate after conversion to the inference policy [22, 23]. Together these methods show why the location, direction, and stage of control matter. Aggregate clipping fractions or mean log-probability gaps can conceal these distinctions. TRIAGE's MoE routing-conditioned analysis further separates a population trend from reliable directional prediction for each token. Checkpoint selection adds another boundary: the full-pipeline FP8 study reports its main GRPO endpoints from the best average checkpoint over 500 steps [11], whereas TRIAGE evaluates stable arms at their final scheduled step. Similar endpoint scores can therefore summarize different trajectory-selection rules.

TRACE tests guided alignment against QAT and TIS+SC in joint low-precision execution; TRIAGE tests recovery beyond TIS in native W4A4. These different baseline sets and precision regimes preclude a direct TRACE–TRIAGE ranking [12, 13].

Complementarity is most persuasive when a component ablation identifies the remaining error. QUADS separates weight alignment from activation compensation, QuRL addresses update visibility alongside clipping control, and KV4-RL changes cache fidelity while retaining TIS [5, 8, 17].

Table 9: Within-study results with their execution, checkpoint, and comparison boundaries.

Study	Controlled comparison	Reported result	Interpretation and limit
TRACE [13] 35B MoE	Joint NVFP4 W/A/KV rollout; TIS+SC uses cap 2 and top-128 reconstruction.	Four-task mean: TRACE 75.3, TIS+SC 74.1, BF16 74.9.	SC selects its best mean checkpoint through 400 steps. Task-wise gains are mixed; aggregate values are shown without uncertainty intervals.
TRIAGE [12] 30B MoE	BF16, native NVFP4, TIS, and full gating/repair recipe. Stable arms use step 1700.	Five-task mean: TRIAGE 70.96, TIS 63.18, BF16 72.41.	Recovery relative to TIS in native W4A4; SC/TIS+SC and KV4 are outside the comparison.
Score Centering [10] 30B MoE	Correction rules share the objective; quantization and lag are tested separately.	With FP4 KV, SC and TIS remain stable; stronger INT8/INT4 stress separates them further.	Quantization curves report training accuracy. Shared-objective controls are not full reproductions of every baseline recipe.
KV4-RL [17] 8B dense	Cache precision varies with TIS retained; all learned policies are evaluated in BF16.	At 400 updates, six-task mean: 52.8 versus BF16's 52.4; cumulative full-iteration speedup 1.27×.	Evaluation-seed variation is not independent training-run variation; this is not a joint W4A4/KV4 comparison.

These examples support the combinations actually tested and motivate a broader question: after one intervention acts, which residual mechanism still limits progress?

6.3. Efficiency Gains and Implementation Costs

An intervention's computational benefit depends on the work required for generation, learning, synchronization, and correction. For serialized phases, iteration time can be decomposed as

$$T_{\text{iter}} = T_{\text{rollout}} + T_{\text{learner}} + T_{\text{sync}} + T_{\text{quant/calib}} + T_{\text{other}}, \tag{12}$$

where synchronization and quantization/calibration costs are separated from generation and learning. Accelerating one phase exposes the others. With overlapping execution, the relevant quantity is the critical path, so the phase times in Equation (12) cannot simply be added.

For fixed workload and remaining phases, rollout fraction f, rollout speedup s, and added work hT_0 give $T/T_0 = (1-f) + f/s + h$ (Figure 12). Overlap, response-length changes, or altered concurrency require a different model.

Each intervention changes a different part of this cost. FP8-RL illustrates the work needed for scale estimation and numerical conversion [4]. Reference calibration adds learner computation; guided rounding adds storage, transport, and state matching; selective precision adds work in the protected region. Table 10 relates these dependencies to the information each method needs.

P3O requires batch-level ratio statistics, while MIPU's acceptance step adds inference-side assessment and possible rollback [21, 23]. Miles and QATFactory emphasize conversion and inference-ready weight transfer as part of the execution contract [25, 26]. TBIK's coordinated reductions

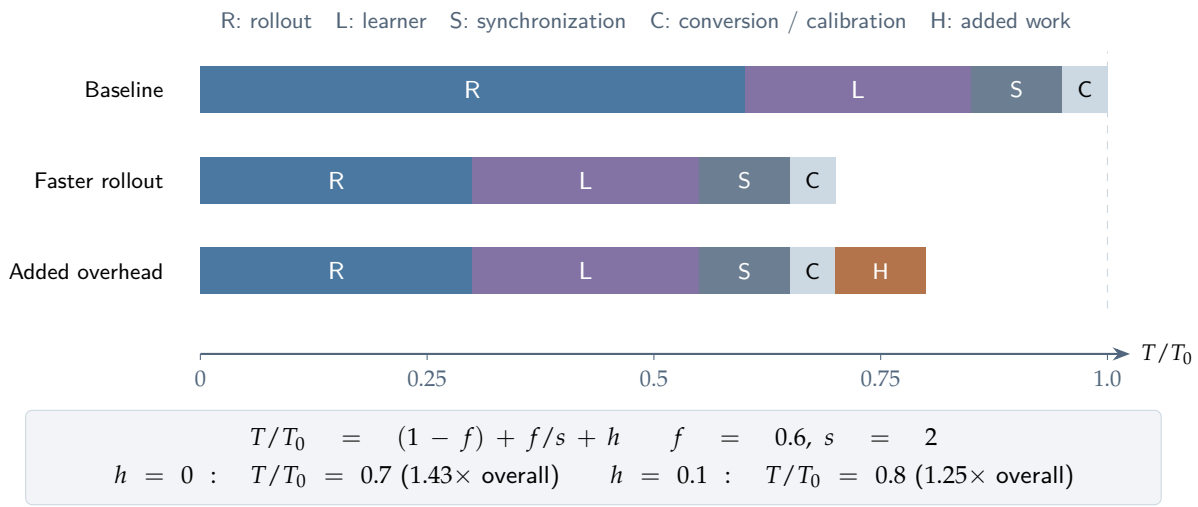


Figure 12: Serialized timing with a faster rollout phase and added overhead. Doubling rollout speed reduces iteration time from T_0 to $0.7T_0$ when rollout accounts for 60% of the baseline. Added work of $0.1T_0$ reduces the overall speedup from $1.43\times$ to $1.25\times$. Constructed illustration under serialized execution.

can carry a kernel-performance cost [28]. Rollout-ResQ's sparse correction is designed for efficient hardware execution, but its simulated experiments do not measure a native FP4 speedup [24].

The useful quantization target depends on the workload as well as the format. QUADS targets native W4A4 expert computation, whereas KV4-RL studies how sustained concurrency and long contexts increase the value of reducing KV processing [8, 17]. Their settings motivate different numerical targets; a bit-width comparison alone does not identify the dominant cost.

Token throughput also depends on the realized workload. A policy that produces shorter responses can appear faster without improving execution per token; one that produces longer responses can incur more work despite a faster kernel. Memory savings can increase concurrency, which changes the workload seen by attention and compute kernels. Serving research such as PagedAttention provides background for why cache management and scheduling affect observed throughput [32], but serving speed alone does not establish RL learning efficiency.

KV4-RL illustrates the difference between generation and training gains. Over 400 iterations on Qwen3-8B-Base, it reports $1.80\times$ cumulative rollout speedup and $1.27\times$ cumulative full-iteration speedup over BF16 rollout [17]. Other phases absorb part of the generation gain. Its common-BF16 evaluation separately compares learned-policy quality under a shared evaluation precision. Both conclusions apply to the study's collection workload.

Quality per budget and cost to a specified quality level connect execution to learning. A remedy may require more learner passes; a faster sampler may change response lengths or the updates needed. Both require the learning trajectory. In overlapped systems, guidance traffic also competes for bandwidth, so isolated overhead need not equal its critical-path cost.

6.4. Evidence Limits and Coverage Gaps

Strong evidence isolates a mechanism within its execution setting. Mathematical reasoning is common; interactive tasks and hardware or kernel transfer have fewer controlled comparisons. Native arithmetic, quantization simulation, and BF16 execution on quantized values remain distinct.

Table 10: Information and execution dependencies that determine implementation cost.

Approach	Information or execution dependency	Consequence for implementation
TIS / AIS / CIS	Behavior probabilities and learner/proximal probabilities	Stored probabilities must include the sampling transformations.
Score Centering	A behavior-weighted score expectation, exact or approximated	Exact evaluation requires distributional information; reconstruction introduces approximation.
Aligned QAT / shared precision	Matching quantization and conversion rules across endpoints	Precision exceptions, kernels, and weight export form part of the configuration.
Guided rounding	Realized rollout-side quantization information	Guidance requires storage, transport, and matching to learner states.
Calibrated Clipping [11]	Higher-precision reference ratios at calibration points	Reference passes add training computation, amortized across calibration intervals.
Batch-adaptive control [21]	Policy-ratio statistics aggregated over the batch	Reduction rules and the sampled batch determine the reliability estimate.
Inference-policy acceptance [23]	Assessment of the synchronized candidate	Candidate evaluation and possible rollback add work beyond learner updates.
Selective KV precision	Low-bit cache execution with protected positions	Protected positions add work whose share depends on the context workload.

Learning comparisons need a budget, checkpoint rule, and uncertainty unit alongside the metric definition in Section 3.4. Repeated decoding measures evaluation variation; independent RL runs expose training variation. Smoothing alone quantifies neither.

Separate W4A4 and KV4 studies do not establish their joint behavior; asynchronous demonstrations do not isolate every precision–lag interaction. Scheduling needs direct low-bit validation, while diffusion needs generation-specific likelihoods before the autoregressive analysis can transfer.

7. Open Problems

The synthesis leaves four connected questions: how to recognize harmful mismatch, compose remedies, identify stateful behavior, and establish transfer across settings.

7.1. Predicting Harmful Mismatch

The diagnostic challenge is to predict lost learning before reward reveals the damage. Ratio tails, conditional score drift, clipping errors, and segment concentration suggest different mechanisms. A useful predictor must distinguish harmful disagreement from legitimate policy movement and from apparent stability achieved by discarding difficult data.

Paired probes localize execution differences, fixed-batch audits test their update effects, and held-out trajectories test their learning consequences. A predictive diagnostic must connect these levels across interventions and changes in the sampled distribution.

7.2. Composing Numerical and Learning Interventions

A component's value depends on the residual error after other components act. Alignment may expose an activation error; estimator correction may change the value of restrictive update control. The combinations in Table 6 offer starting points. Guided rounding with score centering, and cache protection with directional control, remain questions for direct study. Component ablations should identify the effect removed, the useful contributions retained, and the additional cost, separating synergy from retuning or one dominant component.

7.3. Precision, Staleness, and Long-Horizon Interaction

A checkpoint need not identify the behavior policy. Responses can cross synchronization boundaries, and retained KV entries encode earlier computations. Applying the separation in Equation (2) therefore requires the relevant history. What is the smallest sufficient behavior record: token probabilities, version identifiers, cache provenance, or additional execution state?

Early numerical perturbations also change the prefixes visited later, beyond the conditional correction at an observed token. Precision allocation across layers, positions, and training phases should therefore follow learning benefit rather than tensor error alone. Multi-turn environments extend this question to tool observations and state transitions.

7.4. Generalization and Comparable Evidence

Portability follows information requirements: actual decoding probabilities for reweighting, distributional information for score reconstruction, and paired states for rounding guidance. An engine or synchronization change can break these dependencies without changing the format.

Component studies need a shared execution path and base objective; system comparisons must preserve each method's essential design under matched resources. Fixed-budget quality and cost to a predeclared target connect progress to runtime, including failure to reach the target. These requirements provide a basis for controlled cross-method evaluation.

8. Conclusion

Low-precision rollout changes the experience supplied to RL and the policy updates computed from it. The four intervention classes locate remedies in computation alignment, numerical fidelity and update visibility, gradient estimation, and update control, complemented by scheduling.

The central distinctions are between numerical accuracy and cross-path agreement, estimation and update control, and faster execution and more efficient learning. Measurement practice connects them: explicit policies, populations, and stages make a discrepancy interpretable; evidence on retained learning and total cost establishes its value. These connections guide combinations that improve a numerical policy while preserving the experience and updates it needs to learn.

A. Surveyed Papers and Section Mapping

Table 11 maps the twenty-nine focal studies to regimes, components, and discussion locations. Years mark first public appearance; bibliography entries may cite later publications. Parentheses give additional discussion locations. Appendix B defines contribution roles.

Table 11: Surveyed papers: numerical regimes, method components, and section mapping. Conference and preprint versions count as one contribution.

Work / first year	Numerical regime	Component(s) and contribution	Discussion
Core method and system studies (19)			
FlashRL / TIS [1] 2025	Higher-precision learner; quantized rollout	**B1**: Recorded behavior probabilities and truncated correction.	**5.1** (1.1; 2.1)
Jet-RL [3] 2026	Unified FP8 learner and rollout paths	**A1; A2**: Unified FP8 precision flow.	**4.1** (6.2)
FP8-RL [4] 2026	W8A8 rollout; FP8 attention / KV; FP8 learner explored in v2	**A2; system**: W8A8 rollout, FP8 attention/KV, and stack integration.	**4.2** (5.1; 6.2; 6.3)
QuRL [5] 2026	INT8 / FP8 rollout	**B2; A2**: Adaptive clipping and update-aware quantization.	**5.2** (3.1; 4.2; 5.4; 6.2)
QaRL [6] 2026	Quantized MoE rollout; aligned learner QAT	**A1; B2**: Rollout-aligned QAT and trust-band updates.	**4.1** (5.2; 5.4; 6.2)
P3O / ESS [21] 2026	BF16 learner / FP8 rollout; dense reasoning models	**B2**: Batch-adaptive ESS cap and regularization.	**5.2** (6.2; 6.3)
AIS [7] 2026	BF16 learner / FP8 rollout; autoregressive and diffusion models	**B1**: Adaptive importance-weighted estimator mixture.	**5.1** (6.1; 6.2)
DRPO [22] 2026	BF16, FP8 rollout-only, and FP8 end-to-end MoE settings	**B2**: Smooth corrective divergence regularization.	**5.2** (6.2)
MIPU [23] 2026	BF16 learner / FP8 rollout; Qwen3-1.7B and 4B	**B2; formulation**: Inference-policy candidate construction and acceptance.	**5.2** (2.1; 6.2; 6.3)
QUADS [8] 2026	NVFP4 MoE rollout; learner weight QAT	**A1; A2**: NVFP4 MoE weight alignment and activation compensation.	**4.1** (4.2; 5.4; 6.2)
ACRL [9] 2026	FP8 rollout; discrepancy-dependent updates	**B2**: Discrepancy-dependent update control.	**5.2** (3.3; 6.2)
HiFloat4 / Rollout-ResQ [24] 2026	Simulated end-to-end HiF4 / MXFP4; rollout residual correction	**A2**: Sparse residual correction for FP4 rollout activations.	**4.2** (3.1; 6.2; 6.3)
Miles v0.1 [25] 2026	FP8 / MXFP8 / NVFP4 RL recipes; precision exceptions	**A1; A2; system**: Shared quantization, synchronization, and precision contracts.	**4.1** (4.2; 6.1; 6.3)
Score Centering [10] 2026	BF16 learner; FP4/INT4 KV stress and separate lag tests	**B1**: Additive behavior-weighted score correction.	**5.1** (2.2; 3.2; 5.4; 6.2)
Full Pipeline FP8 RL [11] 2026	FP8 learner and rollout; calibration against BF16	**B2; system**: FP8 learner/rollout and calibrated clipping.	**5.2** (3.2; 6.2; 6.3)

Continued on next page

Table 11 continued: Surveyed papers

Work / first year	Numerical regime	Component(s) and contribution	Discussion
QATFactory / QARL [26] 2026	Native NVFP4 rollout; simulated-quantization BF16 training	**A1; system**: Deployment-aligned QARL and inference-ready weight synchronization.	**4.1** (6.2; 6.3)
TRIAGE [12] 2026	Native W4A4 forward; documented BF16 KV; synchronous RL	**B2**: Direction-aware control for native NVFP4.	**5.2** (3.3; 6.2)
TRACE [13] 2026	NVFP4 expert W4A4 + softmax KV4; guided QAT, BF16 backward	**A1**: Rollout-guided FP4 rounding for MoE.	**4.1** (3.1; 6.2)
KV4-RL [17] 2026	BF16 learner; INT4 KV with bounded INT8 protection	**A2; system**: INT4 KV with selective INT8 protection.	**4.2** (5.1; 6.2; 6.3)
Supporting formulation, theory, and general TIM studies (9)			
R3 [15] 2025	MoE routing; general execution mismatch	**Support; A1**: MoE routing alignment.	**3.1** (4.1)
FP16-Mismatch [16] 2025	BF16 / FP16 numerical comparison	**Support; A2**: BF16/FP16 precision-selection evidence.	**3.1**
TBIK / Deterministic Inference [28] 2025	BF16 / FP32; matched reduction order across TP sizes	**Support; A1**: Cross-TP deterministic kernels as an alignment reference.	**3.1** (4.1; 6.3)
MiniRL [14] 2025	Hybrid-engine RL; includes FP8-rollout experiments	**Support; B1/B2**: Policy roles and mismatch-aware formulation; FP8 experiments.	**2.1** (1.2; 5.1)
Beyond Precision [18] 2026	General TIM; optimization scheduling	**Support; B3**: Optimization schedule under general TIM.	**5.3** (3.3)
DPPO [29] 2026	General LLM trust-region and cross-engine mismatch evidence	**Support; B2**: Divergence-based trust regions under general TIM.	**5.2** (3.2)
VeXact / Diagnosing TIM [19] 2026	Matched-execution diagnostic reference	**Diagnostic**: Zero-mismatch diagnostic reference.	**3.4**
CIS [20] 2026	BF16 cross-engine mismatch	**Support; B1**: Confidence-dependent correction under BF16 cross-engine mismatch.	**5.1** (3.2)
Baselines and Score Centering [27] 2026	Sequence- and token-level regularized importance sampling	**Support; B1**: Granularity-dependent roles of baselines and score centering.	**5.1** (5.4)
Exploration boundary case (1)			
QeRL [2] 2025	NVFP4 with LoRA and adaptive noise	**Boundary**: NVFP4 with LoRA and adaptive noise; exploration setting.	**Box 1**

B. Source Selection and Version Handling

Literature identification. This narrative review identifies twenty-nine focal contributions on quantized rollout, low-precision RL, and training–rollout mismatch. Literature identification combined targeted arXiv metadata searches through October 20, 2026, review of primary papers and official publication records, and backward citation tracing.

Inclusion and exclusion. Focal studies contribute a direct quantized-rollout or low-precision RL intervention, a mechanism explaining that setting, or an exploration method in which quantization changes the sampling distribution. General quantization and RL background is cited selectively. Papers devoted only to low-precision pretraining, unrelated generation modalities, or alternative rollout acceleration do not become focal entries solely because they use the same precision format.

Versions and names. The catalog separates first-publication dates, venues, and reviewed versions. QeRL, QuRL, and QaRL use formal publication records. Aliases, conference/preprint versions, and presentation changes count once. Version-unspecified arXiv links remain distinct from pinned reviewed sources because revisions can change the supported claims.

Attribution of evidence. Method claims and observations are attributed to primary sources. The likelihood, drift, and clipping identities explain mechanisms under the stated assumptions; they are distinct from experimental findings. Figures and tables are original syntheses, and constructed examples are labeled separately from published measurements. The companion evidence collection links observations to reviewed sources.

Contribution roles and empirical scope. Core, supporting, and boundary roles describe relevance to the synthesis, independently of experimental quality; evidence domains identify the settings examined. All empirical results come from the cited studies, without independent experimental reproduction. Training-run replication, negative results, and cross-hardware reproduction were not systematically extracted. Versioned records identify the source supporting each observation.

Table 12 summarizes the reviewed settings and contribution roles.

Table 12: Coverage across independent setting axes. Joint regimes require direct evaluation.

Dimension	Represented in the focal collection	Boundary of the synthesis
Intervention	A1/A2 numerical design; B1/B2 learning rules; complementary scheduling.	Direct low-bit evidence is thinner for B3 than for the other classes.
Numerical interface	Weight/activation quantization, KV storage, scales, rounding, routing, and synchronization.	An interface-level result does not establish full-network agreement.
Precision	FP8, FP4/NVFP4, INT8/INT4; BF16/FP16 comparisons provide context.	Format labels omit scale granularity, precision exceptions, and native versus simulated arithmetic.
Model and adaptation	Dense and MoE models; low-rank and full-parameter updates appear in different studies.	Architecture and adaptation are separate axes; cross-regime transfer is not assumed.
Collection	Snapshot-based hybrid execution, policy lag, and quota-based partial collection.	Stateful provenance and fully overlapped execution require additional behavior and timing semantics.
Learning and generation	PPO/GRPO-style autoregressive RL; AIS also studies diffusion generation.	This survey's likelihood equations are autoregressive; denoising requires its own formulation.
Contribution role	Nineteen core studies, nine supporting studies, and one exploration boundary.	Role identifies relevance to the argument, independently of experimental strength.

C. Evidence, Assumptions, and Transfer

Transfer depends on the relationship between sampled behavior and learner computation. Parameter versions, numerical paths, decoding, and retained state determine the behavior law; learner, proximal, and reference policies define separate loss roles. Changing execution can alter a correction's meaning even at the same nominal precision.

Each approximation leaves a specific residual: prefix selection after conditional correction, truncation bias, unmatched execution after alignment, or reconstruction error in an expectation. A tested combination supports its reported estimator and regime; transfer requires the assumptions responsible for that result to remain valid.

Conditional and complete-response correction. Equation (3) corrects actions at a fixed prefix. A response-level expectation also requires correction of how frequently that prefix is reached.

For comparison, the full-response likelihood ratio is

$$W_\theta(y \mid x) = \frac{P_\theta(y \mid x)}{Q(y \mid x)} = \prod_{t=1}^{T} \frac{p_\theta(y_t \mid h_t)}{q_{k(t)}(y_t \mid h_t)}. \tag{13}$$

Here P_θ and Q are response laws under a common stopping convention, including end-of-sequence decisions when applicable. Denominator factors must be actual conditional probabilities. Unobserved execution randomness must be marginalized or included through an extended likelihood over the corresponding execution events.

For a fixed behavior law and a parameter-independent reward R, the usual likelihood-ratio identity gives

$$\nabla_\theta \mathbb{E}_{P_\theta}[R] = \mathbb{E}_Q\left[W_\theta R \sum_{t=1}^{T} s_\theta(y_t, h_t)\right] \tag{14}$$

under support, integrability, and differentiation assumptions. The product illustrates why exact response-level correction can have high variance; multiplying only the current token ratio into an arbitrary sampled advantage is a different estimator. Nor does Equation (14) directly justify reusing group-normalized advantages computed under Q. Score Centering explicitly distinguishes sequence-level importance sampling from practical token-level corrections [10].

Figure 13 contrasts matching second-token conditionals with unequal prefix masses, which the full-response ratio also corrects.

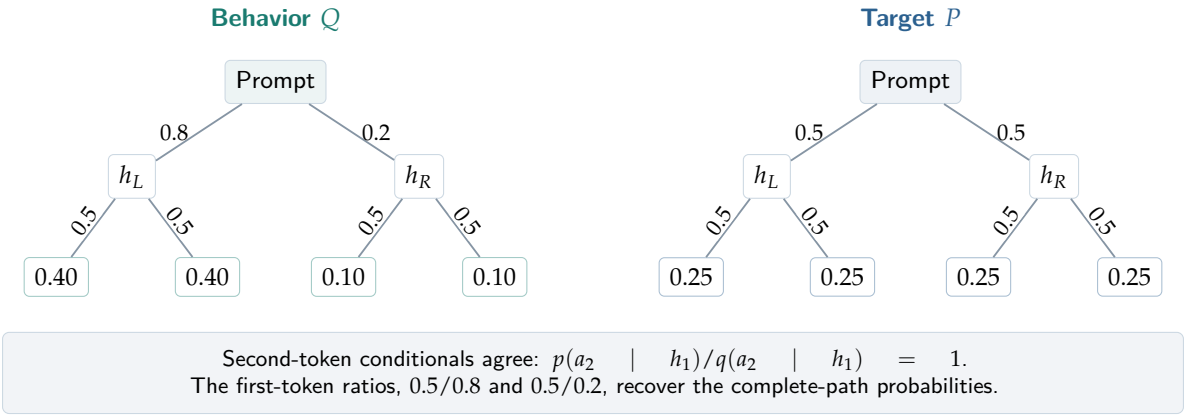


Figure 13: Conditional and response-level correction in a two-step probability tree. Leaf values are complete-response probabilities. Equal second-token conditionals coexist with different prefix masses; the product ratio restores the four target probabilities of 0.25. Constructed two-step example.

D. Terminology

Table 13: Terminology and component abbreviations used in the text and figures.

Term	Meaning
TIM	Training–inference mismatch; the survey emphasizes the learner–rollout interface.
W / A / KV	Weights, activations, and key–value cache; distinct numerical targets.
MoE / LoRA	Mixture-of-experts architecture / low-rank adaptation; architecture and adaptation are separate dimensions.
FP8 / FP4 / NVFP4	Floating-point format families with eight or four bits; NVFP4 is a particular four-bit representation. Scaling, accumulation, and exceptions also determine execution.
INT8 / INT4	Integer quantization at eight or four bits, with scales and execution semantics specified separately.
QAT	Quantization-aware training: the learner forward path models a quantized representation; this need not reproduce native kernel arithmetic.
TIS / MIS	Truncated / masked importance sampling; bounded or discarded corrections alter the estimator.
AIS / CIS	Adaptive / calibrated importance sampling: estimator-mixture adaptation and confidence-dependent truncation, respectively.
ACR / UAQ	QuRL's Adaptive Clipping Range / Update-Aware Quantization; B2 update control and A2 update visibility.
TBPO / RAC	QaRL's Trust-Band Policy Optimization / QUADS's Residual Activation Compensation; distinct objective and numerical components.
PPO / GRPO	Proximal Policy Optimization / Group Relative Policy Optimization. Their loss roles require behavior and proximal policies to be distinguished.
ESS / P3O	Effective sample size / Policy-on Policy-off Policy Optimization; batch-ratio reliability controls update influence and regularization.
DPPO / DRPO	Divergence Proximal / Divergence Regularized Policy Optimization; divergence-based masking versus smooth corrective regularization.
MIPI / MIPU	Monotonic Inference Policy Improvement / Update; inference-policy objective and practical candidate construction/acceptance.
Rollout-ResQ	Sparse residual correction of rollout activation quantization; the reviewed HiFloat4 experiments simulate FP4.
TBIK	Tree-Based Invariant Kernels; matched reduction order across tensor-parallel sizes.
Partial rollout	Collection in which a response can span collection boundaries; it does not by itself imply concurrent generation and optimization.
Cache provenance	Parameter versions and numerical execution paths that produced retained cache entries.
Behavior / proximal / reference	Actual sampling law / update-ratio anchor / KL-regularization target; see Table 1.

E. Reported Mismatch Measurements

Table 14 summarizes selected diagnostic evidence from every core method/system study. It distinguishes explicit policy-distance definitions, control variables, local numerical probes, and learning outcomes. Definitions follow each reviewed source, with unspecified conventions identified in the table. The companion repository links the complete source records.

Table 14: Selected mismatch measurements and comparison conditions across the nineteen core studies. Source locations refer to the reviewed versions.

Study	Reported observable and location	Comparison boundary
FlashRL / TIS [1]	Cross-engine KL (Fig. 3); TIS uses sampled-action ratios.	The plotted KL direction and reduction are not specified in the caption.
Jet-RL [3]	Length- and task-conditioned learning curves (§ 3.2).	Outcome evidence; these curves do not isolate a numerical policy distance.
FP8-RL [4]	Rollout-to-training mismatch KL (§ 2.2.1; Fig. 12).	Direction is explicit; estimator and reduction require further implementation detail.
QuRL [5]	Behavior-to-proximal KL; maximum proximal/behavior ratio (Fig. 3).	Average discrepancy and rare large weights are distinct; neither is the reference KL penalty.
QaRL [6]	Proximal-ratio and mismatch-weight distributions (Fig. 4); response trust bands.	Keep the two ratios separate; geometric-mean response ratios differ from likelihood products.
P3O / ESS [21]	Valid-token normalized ESS, using current/behavior ratios (Eq. 11).	Measures concentration; includes policy change and does not identify numerical error alone.
AIS [7]	Mean absolute log gap; capped-weight ESS; advantage-weight variance amplification (§ 3.1).	These govern distinct gates; capped and raw ESS are different observables.
DRPO [22]	Sampled-action Binary-TV geometry; learning and entropy trajectories (§ 3; App. D.4).	The constraint is not full-vocabulary TV or, by itself, an isolated TIM measurement.
MIPU [23]	Sampled-token mismatch K3; inference-gap proxy and rollback rate (§ 5).	KL estimation and candidate acceptance assess different stages of an update.
QUADS [8]	Maximum log gap; layer-wise MoE L2 discrepancy (Figs. 6–7).	Local tensor error and policy-level extremes have different scales and comparison populations.
ACRL [9]	Absolute probability gap, response/group averaged relative to baseline X (Eqs. 8–12).	Distinct from log gaps; the baseline-relative controller does not define a universal tolerance.
HiFloat4 / Rollout-ResQ [24]	Residual sparsification MSE/RMSE and precision ablations (Fig. 4; § 5).	Tensor reconstruction error should not be relabeled a learner–sampler policy distance.
Miles v0.1 [25]	Same-token log-probability checks; reference-run KL; exact-mode zero difference (§ 3.1; § 5.3; Fig. 5).	Different modes use different comparisons; exact sampled-token scores do not establish full-distribution equality.

Continued on next page

Table 14 continued

Study	Reported observable and location	Comparison boundary
Score Centering [10]	Conditional score-mean analysis; controlled learning curves under noise, quantization, and lag (§§ 3–5).	Distinguish the analytical drift term from logged measurements and deliberately amplified stress.
Full Pipeline FP8 RL [11]	Probability/ratio NMAE against BF16 shadows; sign-separated over-clipping (§ 3).	Counterfactual branch errors differ from ordinary clipping rates and fixed-snapshot TIM.
QATFactory / QARL [26]	QARL learning curves and exported-policy quality (§ 4.6).	Held-out teacher KL elsewhere in the paper belongs to QAD, not an online RL TIM measurement.
TRIAGE [12]	Mean absolute gap, signed tails, directional masses, and localization (§ 3; App. B).	30B characterization is routing-conditioned; its raw magnitude is not directly comparable with ordinary 4B gaps.
TRACE [13]	Log-gap maxima/minima; relative excess train–rollout activation MSE (Figs. 4, 11).	Extremes differ from means; excess MSE uses BF16 cross-path MSE as its denominator.
KV4-RL [17]	Shared-prefix full-distribution BF16-to-quantized KL (Eq. 5.2; App. B.5).	Same checkpoint and pre-decoding probabilities isolate conditional KV distortion; trajectory shift remains outside the metric.

Measurement definitions and reproducibility. The studies differ in execution controls, reduction rules, and uncertainty units. Table 15 organizes these choices into the context needed to interpret and reproduce a reported measurement.

Table 15: Execution controls, statistical reductions, and uncertainty units used in mismatch diagnostics.

Measurement choice	Source practice or example	Required interpretation context
Execution-only comparison	KV4-RL fixes checkpoint and BF16 prefixes for full-distribution KL [17].	Prefix source; raw versus decoded probabilities; routes and cache state; KL direction.
Update-time comparison	TRIAGE replays an identical batch before/after an update with generation log-probabilities held fixed [12].	Learner versions; pre/post stage; fixed advantages and masks; route replay.
Collection-time record	CIS logs generation scores, start-of-update learner scores and rollout versions [20].	Decode versus prefill rescoring; version lag; stored numeric precision and sample selection.
Batch reliability	P3O aggregates raw-ratio moments; AIS uses truncated weights for its ESS gate [7, 21].	Weight definition, cap, valid-token count, and cross-rank reduction.
Decision disagreement	Full-pipeline FP8 compares clipping decisions with BF16 shadow ratios [11].	Reference path; advantage sign; boundary; ordinary versus counterfactual clipping.
Uncertainty and time axis	KV4-RL uses response-cluster bootstrap intervals for its paired fidelity probe [17].	Independent unit: token, response, evaluation seed, or training run; updates, sampled tokens, or elapsed time.

Policy pairs and paired execution. Learner–sampler, current–proximal, and learner–reference KL concern behavior disagreement, policy movement, and regularization. An *actor KL* label needs its policy pair, direction, and estimator. QATFactory's teacher–student KL evaluates QAD fidelity, separately from online QARL mismatch [26].

To isolate an update's motion, replay the same prefixes before and after the update with the generation scores held fixed, as in TRIAGE's directional audit [12]. To isolate numerical execution, also match the sampling-snapshot parameters and specify kernels, routes, and cache state. A current-learner/old-behavior comparison otherwise combines execution differences and lag.

Measurement population and aggregation. Token averaging weights responses by length; response averaging gives them equal weight. For lengths 2 and 8 with constant absolute gaps 1 and 0.1, the means are 0.28 and 0.55. Global sums and counts define the token mean; P3O likewise reduces ESS moments across workers [21].

Selection can lower a diagnostic without improving agreement on the original batch. Interpretation therefore requires both a fixed valid-token population and the intervention-eligible subset, together with its occupancy and coefficient- or advantage-weighted mass. TRIAGE's separate marginal statistics, gradient-bearing occupancy, and directional masses make this distinction explicit [12]. Raw and capped ESS similarly need the weight definition and cap.

Numerical evaluation. Stored log-probability precision can impose a measurement floor. Saturation and clamping affect log-odds and binary divergences near probability one, as CIS demonstrates in a masking comparison [20]. Logit comparisons additionally require a shared offset convention, since adding the same constant to every vocabulary logit leaves all token probabilities unchanged.

Linking diagnostics to learning outcomes. An informative evaluation links the central gap and its tails to the targeted estimator or update effect, then to held-out quality and total cost. The mechanism determines the additional observable: paired activation error for rounding alignment, weight moments for estimation, or sign-conditioned branch disagreement for clipping calibration.

References

[1] Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, Jianfeng Gao. On the Rollout-Training Mismatch in Modern RL Systems. *OPT 2025: Optimization for Machine Learning*, 2025. https://www.opt-ml.org/papers/2025/paper116.pdf.

[2] Wei Huang, Yi Ge, Shuai Yang, Yicheng Xiao, Huizi Mao, Yujun Lin, Hanrong Ye, Sifei Liu, Ka Chun Cheung, Hongxu Yin, Yao Lu, Xiaojuan Qi, Song Han, Yukang Chen. QeRL: Beyond Efficiency - Quantization-enhanced Reinforcement Learning for LLMs. *International Conference on Learning Representations*, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/a15a2ece7f0663d1ba7db91103ac61c9-Abstract-Conference.html.

[3] Haocheng Xi, Charlie Ruan, Peiyuan Liao, Yujun Lin, Han Cai, Yilong Zhao, Shuo Yang, Kurt Keutzer, Song Han, Ligeng Zhu. Jet-RL: Enabling On-Policy FP8 Reinforcement Learning with Unified Training and Rollout Precision Flow. *arXiv preprint*, 2026. https://arxiv.org/abs/2601.14243v2.

[4] Zhaopeng Qiu, Shuang Yu, Jingqi Zhang, Shuai Zhang, Xue Huang, Jingyi Yang, Junjie Lai. FP8-RL: A Practical and Stable Low-Precision Stack for LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2601.18150v2.

[5] Yuhang Li, Reena Elangovan, Xin Dong, Priyadarshini Panda, Brucek Khailany. QuRL: Low-Precision Reinforcement Learning for Efficient Reasoning. *International Conference on Learning Representations*, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/6b4880ced785077159b340710ceb7681-Abstract-Conference.html.

[6] Hao Gu, Hao Wang, Jiacheng Liu, Lujun Li, Qiyuan Zhu, Bei Liu, Binxing Xu, Lei Wang, Xintong Yang, Sida Lin, Sirui Han, Yike Guo. QaRL: Rollout-Aligned Quantization-Aware RL for Fast and Stable Training under Training–Inference Mismatch. *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. https://aclanthology.org/2026.findings-acl.841/.

[7] Jiajun Zhou, Wei Shao, Lingchao Zheng, Yuwei Fan, Ngai Wong. AIS: Adaptive Importance Sampling for Quantized RL. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.13907v1.

[8] Zhengyang Zhuge, Hao Yu, Xin Wang, Zheng Li, Yizhong Cao, Dayiheng Liu, Jianwei Zhang. QUADS: Stabilizing NVFP4 Reinforcement Learning for MoE via QUantization-error Alignment across Dual Sides. *arXiv preprint*, 2026. https://arxiv.org/abs/2607.15810v1.

[9] Wenwu Fan, Qihong Lin, Zhijie Xia, Zhuo Zheng, Sihao Wang, Qiang Chen, Liangsheng Zhu. ACRL: Adaptive Control of Training-Inference Discrepancy for Stable Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2607.24062v1.

[10] Martin Marek, Max Ryabinin. Score Centering Stabilizes Off-policy Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.20807v1.

[11] Fanchao Chen, Ziheng Jiang, Ziyun Wei, Zheng Zhong, Du Li, Chi Zhang, Haibin Lin, Shivaram Venkataraman. Towards Full Pipeline FP8 Reinforcement Learning for LLMs. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.22870v1.

[12] Zhen Li, Shuai Zhang, Yanggan Gu, Yiming Zhang, Yang Yu, Mingfa Feng, Congkai Xie, Shuang Yu, Junjie Lai, Hongxia Yang. TRIAGE: Direction-Aware Mismatch Stabilization of Native NVFP4 Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.07043v2.

[13] Xin Wang, Hao Yu, Zhengyang Zhuge, Bochao Mao, Zheng Li, Junda Feng, Yuyan Luo, Yi Zhang, Yizhong Cao, Mi Zhang, Dayiheng Liu, Jianwei Zhang. TRACE: Rollout-Guided Quantization-Aware Training for FP4 Reinforcement Learning of MoE Language Models. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.07767v2.

[14] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, Junyang Lin. Stabilizing Reinforcement Learning with LLMs: Formulation and Practices. *arXiv preprint*, 2025. https://arxiv.org/abs/2512.01374v3.

[15] Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, Fuli Luo. Stabilizing MoE Reinforcement Learning by Aligning Training and Inference Routers. *arXiv preprint*, 2025. https://arxiv.org/abs/2510.11370v2.

[16] Penghui Qi, Zichen Liu, Xiangxin Zhou, Tianyu Pang, Chao Du, Wee Sun Lee, Min Lin. Defeating the Training-Inference Mismatch via FP16. *arXiv preprint*, 2025. https://arxiv.org/abs/2510.26788v1.

[17] Sihwan Park, Sung-Yub Kim, Doohyuk Jang, Eunho Yang. KV4-RL: Unlocking Faster Asynchronous Rollouts for RLVR with INT4 KV Cache Quantization. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.14448v1.

[18] Yaxiang Zhang, Yingru Li, Jiacai Liu, Jiawei Xu, Ziniu Li, Qian Liu, Haoyuan Li. Beyond Precision: Training-Inference Mismatch is an Optimization Problem and Simple LR Scheduling Fixes It. *arXiv preprint*, 2026. https://arxiv.org/abs/2602.01826v1.

[19] Tianle Zhong, Neiwen Ling, Yifan Pi, Zijun Wei, Tianshu Yu, Geoffrey Fox, Peng Wu, Xiao Yu. Diagnosing Training Inference Mismatch in LLM Reinforcement Learning via a Zero-Mismatch Reference. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.14220v2.

[20] Tianrun Yu, Kaixiang Zhao, Shangzhe Li, Yuxiao Yang, Porter Jenkins, Weitong Zhang, Taylor W. Killian. Rethinking Training-Inference Mismatch in LLM Reinforcement Learning: Where It Arises and How to Correct It. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.32444v1.

[21] Rasool Fakoor, Murdock Aubry, Nicholas Stranges, Alexander J. Smola. Let the Batch Decide, On- or Off-Policy: Effective Sample Size for RL Post-Training. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.12380v2.

[22] Jiarui Yao, Xiangxin Zhou, Penghui Qi, Wee Sun Lee, Liefeng Bo, Tianyu Pang. Rethinking the Divergence Regularization in LLM RL. *arXiv preprint*, 2026. https://arxiv.org/abs/2606.09821v1.

[23] Jing Liang, Hongyao Tang, Yi Ma, Yancheng He, Weixun Wang, Xiaoyang Li, Ju Huang, Wenbo Su, Jinyi Liu, Yan Zheng, Jianye Hao, Bo Zheng. The Mirage of Optimizing Training Policies: Monotonic Inference Policies as the Real Objective for LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2606.29526v1.

[24] Hei Yi Mak, Shadan Golestan, Hoang Le, Mehran Taghian Jazi, Yunke Peng, Yaoyuan Wang, Yao Wang, Junsong Wang, Tianchi Hu, Fengchen He, Guipeng Hu, Tanzila Rahman, Anandharaju Durai Raju. HiFloat4 Format for End-To-End Reinforcement Learning Post-Training of Large Language Models. *arXiv preprint*, 2026. https://arxiv.org/abs/2607.26515v1.

[25] RadixArk, Tom Chen, Mao Cheng, Shi Dong, Kangrui Du, Yanbin Jiang, Jiajun Li, Yiming Li, Tao Lin, Yusheng Su, Andy Ye, Yueming Yuan, Zhichen Zeng. Miles v0.1: Production-Level Post-Training. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.08368v1.

[26] Weili Xu, Jisen Li, Yuqing Jian, Chenxi Li, Zhizhou Sha, Yifan Yu, Qingyang Wu, Chenfeng Xu, Zhongzhu Zhou, Tianyi Zhang, Ben Athiwaratkun. QATFactory: A Versatile, Deployment-Aligned Framework for Quantization-aware Training and Distillation of LLMs. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.39223v2.

[27] Shuo Han, Ziqing Hu. Baselines and Score Centering in Policy Gradient Estimation. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.22263v1.

[28] Ziyang Zhang, Xinheng Ding, Jiayi Yuan, Rixin Liu, Huizi Mao, Jiarong Xing, Zirui Liu. Deterministic Inference across Tensor Parallel Sizes That Eliminates Training-Inference Mismatch. *arXiv preprint*, 2025. https://arxiv.org/abs/2511.17826v2.

[29] Penghui Qi, Xiangxin Zhou, Zichen Liu, Tianyu Pang, Chao Du, Min Lin, Wee Sun Lee. Rethinking the Trust Region in LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2602.04879v3.

[30] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint*, 2017. https://arxiv.org/abs/1707.06347.

[31] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint*, 2024. https://arxiv.org/abs/2402.03300.

[32] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv preprint*, 2023. https://arxiv.org/abs/2309.06180.

[33] Zhiwei Hao, Jianyuan Guo, Li Shen, Yong Luo, Han Hu, Guoxia Wang, Dianhai Yu, Yonggang Wen, Dacheng Tao. Low-Precision Training of Large Language Models: Methods, Challenges, and Opportunities. *arXiv preprint*, 2025. https://arxiv.org/abs/2505.01043.

[34] Rohan Surana, Gagan Mundada, Xunyi Jiang, Chuhan Wang, Zhenwei Tang, Difan Jiao, Zihan Huang, Yuxin Xiong, Junda Wu, Sheldon Yu, Xintong Li, Raghav Jain, Nikki Kuang, Sizhe Zhou, Bowen Jin, Zhendong Chu, Tong Yu, Ryan Rossi, Kuan-Hao Huang, Jingbo Shang, Jiawei Han, Julian McAuley. Generate, Filter, Control, Replay: A Comprehensive Survey of Rollout Strategies for LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.02913.

[35] Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, Che Jiang, Yuchen Fan, Kai Tian, Guoli Jia, Pengfei Li, Yu Fu, Xingtai Lv, Yuchen Zhang, Sihang Zeng, Shang Qu, Haozhan Li, Shijie Wang, Yuru Wang, Xinwei Long, Fangfu Liu, Xiang Xu, Jiaze Ma, Xuekai Zhu, Ermo Hua, Yihao Liu, Zonglin Li, Huayu Chen, Xiaoye Qu, Yafu Li, Weize Chen, Zhenzhao Yuan, Junqi Gao, Dong Li, Zhiyuan Ma, Ganqu Cui, Zhiyuan Liu, Biqing Qi, Ning Ding, Bowen Zhou. A Survey of Reinforcement Learning for Large Reasoning Models. *arXiv preprint*, 2025. https://arxiv.org/abs/2509.08827.